

Computing Inconsistency Measures Under Differential Privacy

SHUBHANKAR MOHAPATRA, University of Waterloo, Canada

AMIR GILAD*, Hebrew University, Israel

XI HE*, University of Waterloo, Canada

BENNY KIMELFELD*, Technion, Israel

Assessing data quality is crucial to knowing whether and how to use the data for different purposes. Specifically, given a collection of integrity constraints, various ways have been proposed to quantify the inconsistency of a database. Inconsistency measures are particularly important when we wish to assess the quality of private data without revealing sensitive information. We study the estimation of inconsistency measures for a database protected under Differential Privacy (DP). Such estimation is nontrivial since some measures intrinsically query sensitive information, and the computation of others involves functions on underlying sensitive data. Among five inconsistency measures that have been proposed in recent work, we identify that two are intractable in the DP setting. The major challenge for the other three is high sensitivity: adding or removing one tuple from the dataset may significantly affect the outcome. To mitigate that, we model the dataset using a conflict graph and investigate private graph statistics to estimate these measures. The proposed machinery includes adapting graph-projection techniques with parameter selection optimizations on the conflict graph and a DP variant of approximate vertex cover size. We experimentally show that we can effectively compute DP estimates of the three measures on five real-world datasets with denial constraints, where the density of the conflict graphs highly varies.

CCS Concepts: • **Security and privacy** → **Data anonymization and sanitization; Privacy-preserving protocols**; • **Information systems** → *Integrity checking*; **Data cleaning**.

Additional Key Words and Phrases: Differential privacy, Inconsistency measures, Integrity constraints

ACM Reference Format:

Shubhankar Mohapatra, Amir Gilad, Xi He, and Benny Kimelfeld. 2025. Computing Inconsistency Measures Under Differential Privacy. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 140 (June 2025), 27 pages. <https://doi.org/10.1145/3725397>

1 Introduction

Differential Privacy (DP) [17] has become the de facto standard for querying sensitive databases and has been adopted by various industry and government bodies [1, 13, 19]. DP offers high utility for aggregate data releases while ensuring strong guarantees on individuals' sensitive data. The laudable progress in DP study, as demonstrated by multiple recent works [26, 35, 64–67], has made it approachable and useful in many common scenarios. A standard DP mechanism adds noise to

*Authors AG, XH, BK have equal contribution and are listed in alphabetical order

Authors' Contact Information: Shubhankar Mohapatra, shubhankar.mohapatra@uwaterloo.ca, University of Waterloo, Waterloo, Canada; Amir Gilad, amirg@cs.huji.ac.il, Hebrew University, Jerusalem, Israel; Xi He, xi.he@uwaterloo.ca, University of Waterloo, Waterloo, Canada; Benny Kimelfeld, bennyk@cs.technion.ac.il, Technion, Haifa, Israel.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/6-ART140
<https://doi.org/10.1145/3725397>

the query output, constrained by a privacy budget that quantifies the permitted privacy leakage. Once the privacy budget is exhausted, no more queries can be answered directly using the database. However, while DP ensures data privacy, it limits users' ability to directly observe or assess data quality, leaving them to rely on the data without direct validation.

The utility of such sensitive data primarily depends on its quality. Therefore, organizations that build these applications spend vast amounts of money on purchasing data from private data marketplaces [39, 57, 60, 63]. These marketplaces build relationships and manage monetary transactions between data owners and buyers. These buyers are often organizations that want to develop applications such as machine learning models or personalized assistants. Before the buyer purchases a dataset at a specific cost, they may want to ensure the data is suitable for their use case, adhere to particular data quality constraints, and be able to profile its quality to know if the cost reflects the quality.

To address such scenarios, we consider the problem of assessing the quality of databases protected by DP. Such quality assessment will allow users to decide whether they can rely on the conclusions drawn from the data or whether the suggested data is suitable for them. To solve this problem, we must tackle several challenges. First, since DP protects the database, users can only observe noisy aggregate statistics, which can be challenging to summarize into a quality score. Second, if the number of constraints is large (e.g., if they were generated with an automatic system [6, 41, 52]), translating each constraint to an SQL COUNT query and evaluating it over the database with a DP mechanism may lead to low utility since the number of queries is large, allowing for only a tiny portion of the privacy budget to be allocated to each query.

Hence, our proposed solution employs *inconsistency measures* [4, 24, 25, 40, 42, 44, 50, 59] that quantify data quality with a single number for all constraints, essentially yielding a *data quality score*. This approach aligns well with DP, as such measures give a single aggregated numerical value representing data quality, regardless of the given number of constraints. As inconsistency measures, we adopt the ones studied by Livshits et al. [44] following earlier work on the topic [4, 24, 50, 59]. This work discusses and studies five measures, including (1) the *drastic measure*, a binary indicator for whether the database contains constraint violations, the (2) *maximal consistency measure*, counting the number of maximal tuple sets for which addition of a single tuple will cause a violation, the (3) *minimal inconsistency measure*, counting the number of minimal tuple sets that violate a constraint, the (4) *problematic measure*, counting the number of constraint violations, the (5) *minimal repair measure*, counting the minimal tuple deletions needed to achieve consistency. These measures apply to various inconsistency measures that have been studied in the literature of data quality management, including functional dependencies, the more general conditional functional dependencies [8], and the more general denial constraints [9]. We show that the first two measures are incompatible for computation in the DP setting (Section 3), focusing throughout the paper on the latter three.

An approach that one may suggest to computing the inconsistency measures in a DP manner is to translate the measure into an SQL query and then compute the query using an SQL engine that respects DP [14, 29, 35, 58]. Specifically relevant is R2T [14], the state-of-the-art DP mechanism for SPJA queries, including self-joins. Nevertheless, when considering the three measures of inconsistency we focus on, this approach has several drawbacks. One of these measures (number of problematic tuples) requires the SQL DISTINCT operator that R2T cannot handle. In contrast, another measure (minimal repair) cannot be expressed at all in SQL, making such engines irrelevant.

Contrasting the first approach, the approach we propose and investigate here models the violations of the integrity constraints as a *conflict graph* and applies DP techniques for graph statistics. In the conflict graph, nodes are tuple identifiers, and there is an edge between a pair of tuples if this pair violates a constraint. Then, each inconsistency measure can be mapped to a specific graph

Table 1. Relative errors for a SQL approach vs our approach to compute the minimal inconsistency measure at $\epsilon = 1$

DP Algorithms	Adult [3]	Flight [48]	Stock [49]
R2T [14]	0.17 ± 0.01	0.12 ± 0.03	123.19 ± 276.73
This work	0.10 ± 0.05	0.10 ± 0.20	0.07 ± 0.08

statistic. Using this view of the problem allows us to leverage prior work on releasing graph statistics with DP [12, 27, 33] and develop tailored mechanisms for computing inconsistency measures with DP.

To this end, we harness graph projection techniques from the state-of-the-art DP algorithms [12] that truncate the graph to achieve DP. While these algorithms have proven effective in prior studies on social network graphs, they may encounter challenges with conflict graphs arising from their unique properties. To overcome this, we devise a novel optimization for choosing the truncation threshold. We further provide a DP mechanism for the minimal repair measure that augments the classic 2-approximation of the vertex cover algorithm [61] to restrict its sensitivity and allow effective DP guarantees with high utility. Our experimental study shows that our novel algorithms prove efficacious for different datasets with various conflict graph sizes and sparsity levels.

Beyond handling the two inconsistency measures that R2T cannot handle, our approach provides considerable advantages even for the one R2T can handle (number of conflicts). For illustration, Table 1 shows the results of evaluating R2T [14] on three datasets with the same privacy budget of 1 for this measure. Though R2T performed well for the Adult and Flight datasets, it reports more than 120% relative errors for the Stock dataset with very few violations. On the other hand, our approach demonstrates strong performance across all three datasets.

The main contributions of this paper are as follows. First, we formulate the novel problem of computing inconsistency measurements with DP for private datasets and discuss the associated challenges, including a thorough analysis of the sensitivity of each measure. Second, we devise several algorithms that leverage the conflict graph and algorithms for releasing graph statistics under DP to estimate the measures that we have determined are suitable. Specifically, we propose a new optimization for choosing graph truncation threshold that is tailored to conflict graphs and augment the classic vertex cover approximation algorithms to bound its sensitivity to 2 to obtain accurate estimates of the measures. Third, we present experiments on five real-world datasets with varying sizes and densities to show that the proposed DP algorithms are efficient in practice. Our average error across these datasets is 1.3%-67.9% compared to the non-private measure.

2 Preliminaries

We begin with some background that we need to describe the concept of inconsistency measures for private databases.

2.1 Database and Constraints

We consider a single-relation schema $\mathcal{A} = (A_1, \dots, A_m)$, which is a vector of distinct attribute names A_i , each associated with a domain $\text{dom}(A_i)$ of values. A database D over $\mathcal{A}$ is associated with a set $\text{tids}(D)$ of *tuple identifiers*, and it maps every identifier $i \in \text{tids}(D)$ to a tuple $D[i] = (a_1, \dots, a_m)$ in $A_1 \times \dots \times A_m$. A database D' is a *subset* of D , denoted $D' \subseteq D$, if D' is obtained from D by deleting zero or more tuples, that is, $\text{tids}(D') \subseteq \text{tids}(D)$ and $D'[i] = D[i]$ for all $i \in \text{tids}(D')$.

Following previous work on related topics [21, 41], we focus on Denial Constraints (DCs) on pairs of tuples. Using the formalism of Tuple Relational Calculus, such a DC is of the form $\forall t, t' \neg (\varphi_1 \wedge \dots \wedge \varphi_k)$ where each φ_j is a comparison $\sigma_1 \circ \sigma_2$ so that: (a) each of σ_1 and σ_2 is either

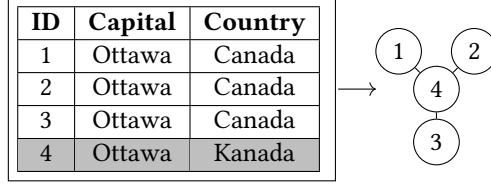


Fig. 1. Toy example dataset to show a worst-case analysis. An additional row may violate all other rows in the dataset (left). Easier analysis can be done by instead converting the dataset into its corresponding conflict graph (right).

$t[A_i]$, or $t'[A_i]$, or a , where A_i is some attribute and a is a constant value, and (b) the operator $\circ$ belongs to set $\{<, >, \leq, \geq, =, \neq\}$ of comparisons. This DC states that there cannot be two tuples t and t' such that all comparisons φ_j hold true (i.e., at least one φ_j should be violated).

The class of DCs of the form that we consider generalizes the class of Functional Dependencies (FDs). An FD has the form $X \rightarrow Y$ where $X, Y \subseteq \{A_1, \dots, A_m\}$, and it states that every two tuples that agree on (i.e., have the same value in each attribute of) X must also agree on Y .

In the remainder of the paper, we denote by Σ the given set of DCs. A database D satisfies Σ , denoted $D \models \Sigma$, if D satisfies every DC in Σ ; otherwise, D violates Σ , denoted $D \not\models \Sigma$.

A common way of capturing the violations of Σ in D is through the *conflict graph* $\mathcal{G}_\Sigma^D$, which is the graph (V, E) , where $V = \text{tids}(D)$ an edge $e = \{i, j\} \in E$ occurs whenever the tuples $D[i]$ and jointly $D[j]$ violate Σ . To simplify the notation, we may write simply $\mathcal{G}$ instead of $\mathcal{G}_\Sigma^D$ when there is no risk of ambiguity.

EXAMPLE 1. Consider a dataset that stores information about capital and country as shown in Figure 1. Assume an FD constraint $\sigma : \text{Capital} \rightarrow \text{Country}$ between attributes capital and country that says that the country of two tuples must be the same if their capital is the same. Assume the dataset has 3 rows (white color) and a neighboring dataset has an extra row (grey color) with the typo in its country attribute. As shown in the right side of Figure 1, the dataset with 4 rows can be converted to a conflict graph with the nodes corresponding to each tuple and edges referring to conflicts between them. The $\mathcal{I}_{\text{MI}}$ measure computes the size of the set of all minimally inconsistent subsets $|\text{MI}_\Sigma(D)|$ (the number of edges in the graph) for this dataset.

2.2 Inconsistency Measures

Inconsistency measures have been studied in previous work [4, 24, 25, 40, 42] as a means of measuring database quality for a set of DCs. We adopt the measures and notation of Livshits et al. [44]. Specifically, they consider five inconsistency measures that capture different aspects of the dataset quality. To define these concepts, we need some notation. Given a database D and a set Σ of anti-monotonic integrity constraints, we denote by $\text{MI}_\Sigma(D)$ the set of all *minimally inconsistent subsets*, that is, the sets $E \subseteq D$ such that $E \not\models \Sigma$ but $E' \models \Sigma$ for all $E' \subsetneq E$. We also denote by $\text{MC}_\Sigma(D)$ the set of all *maximal consistent subsets* of D ; that is, the sets $E \subseteq D$ such that $E \models \Sigma$ and $E' \not\models \Sigma$ whenever $E \subsetneq E' \subseteq D$.

DEFINITION 1 (INCONSISTENCY MEASURES [44]). Given a database D and a set of DCs Σ , the inconsistency measures are defined as follows:

- *Drastic measure:* $\mathcal{I}_D(D, \Sigma) = 1$ if $D \models \Sigma$ and 0 otherwise.
- *Minimal inconsistency measure:* $\mathcal{I}_{\text{MI}}(D, \Sigma) = |\text{MI}_\Sigma(D)|$.
- *Problematic measure:* $\mathcal{I}_P(D, \Sigma) = |\cup \text{MI}_\Sigma(D)|$.

- *Maximal consistency measure:* $\mathcal{I}_{MC}(D, \Sigma) = |\mathcal{MC}_{\Sigma}(D)|$ ¹.
- *Optimal repair measure:* $\mathcal{I}_R(D, \Sigma) = |D| - |D_R|$, where $|D_R|$ is the largest subset $D_R \subseteq D$ such that $D_R \models \Sigma$.

Observe that inconsistency measures also have a graphical interpretation for the conflict graph $\mathcal{G}_{\Sigma}^D$. For instance, the drastic measure $\mathcal{I}_D(D, \Sigma)$ corresponds to a binary indicator for whether there exists an edge in $\mathcal{G}_{\Sigma}^D$. We summarize the graph interpretation of these inconsistency measures in Table 2.

2.3 Differential Privacy

Differential privacy (DP) [17] aims to protect private information in the data. In this work, we consider the unbounded DP setting where we define two neighboring datasets, D and D' (denoted by $D \approx D'$) if D' can be transformed from D by adding or removing one tuple in D .

DEFINITION 2 (DIFFERENTIAL PRIVACY [17]). *An algorithm $\mathcal{M}$ is said to satisfy ϵ -DP if for all $S \subseteq \text{Range}(\mathcal{M})$ and for all $D \approx D'$,*

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{M}(D') \in S].$$

The privacy cost is measured by the parameters ϵ , often called the *privacy budget*. The smaller ϵ is, the stronger the privacy is. Complex DP algorithms can be built from the basic algorithms following two essential properties of differential privacy:

PROPOSITION 1 (DP PROPERTIES [15, 16]). *The following hold.*

- (1) **(Sequential composition)** *If $\mathcal{M}_i$ satisfies ϵ_i -DP, then the sequential application of $\mathcal{M}_1, \mathcal{M}_2, \dots$, satisfies $(\sum_i \epsilon_i)$ -DP.*
- (2) **(Parallel composition)** *If each $\mathcal{M}_i$ accesses disjoint sets of tuples, they satisfy $(\max_i \epsilon_i)$ -DP together.*
- (3) **(Post-processing)** *Any function applied to the output of an ϵ -DP mechanism $\mathcal{M}$ also satisfies ϵ -DP.*

Many applications in DP require measuring the change in a particular function's result over two neighboring databases. The supremum over all pairs of neighboring databases is called the *sensitivity* of the function.

DEFINITION 3 (GLOBAL SENSITIVITY [18]). *Given a function $f : \mathcal{D} \rightarrow \mathbb{R}$, the sensitivity of f is*

$$\Delta_f = \max_{D' \approx D} |f(D) - f(D')|. \quad (1)$$

Laplace mechanism. The Laplace mechanism [18] is a common building block in DP mechanisms and is used to get a noisy estimate for queries with numeric answers. The noise injected is calibrated to the query's global sensitivity.

DEFINITION 4 (LAPLACE MECHANISM [18]). *Given a database D , a function $f : \mathcal{D} \rightarrow \mathbb{R}$, and a privacy budget ϵ , the Laplace mechanism $\mathcal{M}_L$ returns $f(D) + v_q$, where $v_q \sim \text{Lap}(\Delta_f/\epsilon)$.*

The Laplace mechanism can answer many numerical queries, but the exponential mechanism can be used in many natural situations requiring a non-numerical output.

Exponential mechanism. The exponential mechanism [46] expands the application of DP by allowing a non-numerical output.

¹We drop "-1" from the original definition [44] for simplicity.

DEFINITION 5 (EXPONENTIAL MECHANISM [46]). Given a dataset D , a privacy budget ϵ , a set Θ of output candidates, a quality function $q(D, \theta_i) \in \mathbb{R}$, the exponential mechanism $\mathcal{M}_{EM}$ outputs a candidate $\theta_i \in \Theta$ with probability proportional to $\exp\left(\frac{\epsilon q(D, \theta_i)}{2\Delta_q}\right)$, where Δ_q is the sensitivity of the quality function q .

Table 2. Summary of Inconsistency Measures, $n = |D| = |\mathcal{G}_\Sigma^D.V|$, $m = |\mathcal{G}_\Sigma^D.E|$, Θ is the candidate set.

Measure	Non-private analysis [44]		DP analysis (this work)		
	Graph Interpretation	Computation	Sensitivity	Computation	Utility
$\mathcal{I}_D$	If exists an edge	$O(\Sigma n^2)$	1	N.A.	N.A.
$\mathcal{I}_{MI}$	No. of edges	$O(\Sigma n^2)$	n	$O(\Sigma n^2 + \Theta m)$	$-\bar{q}_{\text{opt}}(D, \epsilon_2)$ $+O(\frac{\epsilon_{\max} \ln \Theta }{\epsilon_1})$
$\mathcal{I}_P$	No. of nodes with positive degrees	$O(\Sigma n^2)$	n	$O(\Sigma n^2 + \Theta m)$	$-\bar{q}_{\text{opt}}(D, \epsilon_2)$ $+O(\frac{\epsilon_{\max} \ln \Theta }{\epsilon_1})$
$\mathcal{I}_{MC}$	No. of maximal independent sets	#P-complete	$O(3^n)$	N.A.	N.A.
$\mathcal{I}_R$	Minimum vertex cover size	NP-hard	1	$O(\Sigma n^2 + m)$	$\bar{I}_R(D, \Sigma)$ $+O(1/\epsilon)$

DP for graphs. When the dataset is a graph $\mathcal{G} = (V, E)$, the standard definition can be translated to two variants of DP [27]. The first is *edge-DP* where two graphs are neighboring if they differ on one edge, and the second is *node-DP*, when two graphs are neighboring if one is obtained from the other by removing a node (and its incident edges). The two definitions offer different kinds of privacy protection. In our work, as we deal with databases and their corresponding conflict graphs, adding or removing a tuple of the dataset translates to node-DP. The corresponding definition of neighboring datasets changes to neighboring graphs where two graphs $\mathcal{G}$ and $\mathcal{G}'$ are called neighboring $\mathcal{G} \approx \mathcal{G}'$ if $\mathcal{G}'$ can be transformed from $\mathcal{G}$ by adding or removing one node along with all its edges in $\mathcal{G}$. Node-DP provides a stronger privacy guarantee than edge-DP since it protects an individual's privacy and all its connections, whereas edge-DP concerns only one such connection.

DEFINITION 6 (NODE SENSITIVITY). Given a function f over a graph $\mathcal{G}$, the sensitivity of f is $\Delta_f = \max_{\mathcal{G}' \approx \mathcal{G}} |f(\mathcal{G}) - f(\mathcal{G}')|$.

The building blocks of DP, such as the Laplace and Exponential mechanisms, also work on graphs by simply substituting the input to a graph and the sensitivity to the corresponding node sensitivity.

Graph projection. Graph projection algorithms refer to a family of algorithms that help reduce the node sensitivity of a graph by truncating the edges and, hence, bounding the maximum degree of the graph. Several graph projection algorithms exist [7, 33], among which the *edge addition* algorithm [12] stands out for its effectiveness in preserving most of the underlying graph structure. The edge addition algorithm denoted by π_θ^Λ , takes as input the graph $\mathcal{G} = \mathcal{G}_\Sigma^D = (V, E)$, a bound on the maximum degree of each vertex (θ), and a stable ordering of the edges (Λ) to output a projected θ -bounded graph denoted by $\mathcal{G}_\theta = \pi_\theta^\Lambda(\mathcal{G})$.

DEFINITION 7 (STABLE ORDERING [12]). A graph edge ordering Λ is stable if and only if given two neighboring graphs $\mathcal{G} = (V, E)$ and $\mathcal{G}' = (V', E')$ that differ by only a node, $\Lambda(\mathcal{G})$ and $\Lambda(\mathcal{G}')$ are consistent in the sense that if two edges appear both in $\mathcal{G}$ and $\mathcal{G}'$, their relative ordering are the same in $\Lambda(\mathcal{G})$ and $\Lambda(\mathcal{G}')$.

The stable ordering of edges, $\Lambda(\mathcal{G})$, can be any deterministic ordering of all the edges E in the $\mathcal{G}$. Such stabling edge ordering can be easily obtained in practice. For example, it could be an ordering (e.g. alphabetical ordering) based on the node IDs of the graph such that in the neighboring dataset

$\mathcal{G}'$, the edges occur in the same ordering as $\mathcal{G}$. The edge addition algorithm starts with an empty set of edges and operates by adding edges in the same order as Λ so that each node has a maximum degree of θ . To simplify the notation, in the remainder of the paper, we drop Λ and denote the edge addition algorithm $\pi_\theta^\Lambda(\mathcal{G})$ as $\pi_\theta(\mathcal{G})$.

3 Inconsistency Measures under DP

Problem Setup. Consider a private dataset D , a set of DCs Σ , and a privacy budget ϵ . For an inconsistency measure $\mathcal{I}$ from the set $\{\mathcal{I}_D, \mathcal{I}_{MI}, \mathcal{I}_P, \mathcal{I}_{MC}, \mathcal{I}_R\}$ (Definition 1), we would like to design an ϵ -DP algorithm $\mathcal{M}(D, \Sigma, \epsilon)$ such that with high probability, $|\mathcal{M}(D, \Sigma, \epsilon) - \mathcal{I}(D, \Sigma)|$ is bounded with a small error.

Sensitivity Analysis. We first analyze the sensitivity of the five inconsistency measures and discuss the challenges to achieving DP.

PROPOSITION 2. *Given a database D and a set of DCs Σ , where $|D| = n$, the following holds: (1) The global sensitivity of $\mathcal{I}_D$ is 1. (2) The global sensitivity of $\mathcal{I}_{MI}$ is n . (3) The global sensitivity of $\mathcal{I}_P$ is n . (4) The global sensitivity of $\mathcal{I}_{MC}$ is exponential in n . (5) The global sensitivity of $\mathcal{I}_R$ is 1.*

Due to space constraints, the proofs for this proposition and other theorems in the paper are in the full version [47].

Inadequacy of $\mathcal{I}_D$ and $\mathcal{I}_{MC}$. We note that two inconsistency measures are less suitable for DP. First, the drastic measure $\mathcal{I}_D$ is a binary measure that outputs 1 if at least one conflict exists in the dataset and 0 otherwise. Due to its binary nature, the measure's sensitivity is 1, meaning adding or removing a single row can significantly alter the result. Adding DP noise to such a binary measure can render it meaningless.

One way to compute the $\mathcal{I}_D$ measure could be to consider a proxy of $\mathcal{I}_D$ by employing a threshold-based approach that relies on $\mathcal{I}_P$ or $\mathcal{I}_{MI}$. For example, if these measures are below a certain given number, we return 0 and, otherwise, 1. A recent work [51] addresses similar problems for synthetic data by employing the exponential mechanism. However, since we focus on directly computing the measures under DP, we leave this intriguing subject for future work.

The $\mathcal{I}_{MC}$ measure that computes the total number of independent sets in the conflict graph has both computational and high sensitivity issues. First, prior work [40] showed that computing $\mathcal{I}_{MC}$ is #P-complete and even approximating it is an NP-hard problem [55]. Even for special cases where $\mathcal{I}_{MC}$ can be polynomially computed (when $\mathcal{G}_\Sigma^D$ is P_4 -free [34]), we show in Proposition 2 that its sensitivity is exponential in the number of nodes of $\mathcal{G}_\Sigma^D$. This significantly diminishes the utility of its DP estimate. Due to these challenges, we defer the study of $\mathcal{I}_D$ and $\mathcal{I}_{MC}$ to future work.

Challenges for $\mathcal{I}_R$, $\mathcal{I}_{MI}$, and $\mathcal{I}_P$. Although the $\mathcal{I}_R$ measure has a low sensitivity of 1 for its output range $[0, n]$, it is an NP-hard problem, and the common non-private solution is to solve a linear approximation that requires solving a linear program [40]. However, in the worst case, this linear program again has sensitivity equal to n (number of rows in the dataset) and may have up to $\binom{n}{2}$ number of constraints (all rows violating each other). Existing state-of-the-art DP linear solvers [28] are slow and fail for such a challenging task. Our preliminary experiments to solve such a linear program timed out after 24 hours with $n = 1000$. For $\mathcal{I}_{MI}$ and $\mathcal{I}_P$ they have polynomial computation costs and reasonable output ranges. However, they still have high sensitivity n . In the upcoming sections 4 and 5, we show that these problems can be alleviated by pre-processing the input dataset as a conflict graph and computing these inconsistency measures as private graph statistics.

Algorithm 1: Graph projection approach for $\mathcal{I}_{\text{MI}}$ and $\mathcal{I}_{\text{P}}$ **Data:** Dataset D , constraint set Σ , candidate set Θ , privacy budgets ϵ_1 and ϵ_2 **Result:** DP inconsistency measure for $\mathcal{I}_{\text{MI}}$ or $\mathcal{I}_{\text{P}}$

- 1 Construct the conflict graph $\mathcal{G}_{\Sigma}^D$
- 2 Sample θ^* from Θ with a ϵ_1 -DP mechanism // Basic EM (Algorithm 2); Optimized EM (Algorithm 3)
- 3 Compute θ^* -bounded graph $\mathcal{G}_{\theta^*} \leftarrow \pi_{\theta^*}(\mathcal{G}_{\Sigma}^D)$ // Edge addition algorithm [12]
- 4 **Return** $f(\mathcal{G}_{\theta^*}) + \text{Lap}(\frac{\theta^*}{\epsilon_2})$ // $f(\cdot)$ returns edge count for $\mathcal{I}_{\text{MI}}$ and the number of nodes with positive degrees for $\mathcal{I}_{\text{P}}$

4 DP Graph Projection for $\mathcal{I}_{\text{MI}}$ and $\mathcal{I}_{\text{P}}$

Computing graph statistics such as edge count and degree distribution while preserving node-differential privacy (node-DP) is a well-explored area [7, 12, 33]. Hence, in this section, we leverage the state-of-the-art node-DP approach for graph statistics to analyze the inconsistency measures $\mathcal{I}_{\text{MI}}$ and $\mathcal{I}_{\text{P}}$ as graph statistics on the conflict graph $\mathcal{G}_{\Sigma}^D$. However, the effectiveness of this approach hinges on carefully chosen parameters. We introduce two optimization techniques that consider the integrity constraints to optimize parameter selection and enhance the algorithm's utility.

4.1 Graph Projection Approach for $\mathcal{I}_{\text{MI}}$ and $\mathcal{I}_{\text{P}}$

A primary utility challenge in achieving node-DP for graph statistics is their high sensitivity. In the worst case, removing a single node from a graph of n nodes can result in removing $(n - 1)$ edges. To mitigate this issue, the state-of-the-art approach [12] first projects the graph $\mathcal{G}$ onto a θ -bounded graph $\mathcal{G}_{\theta}$, where the maximum degree is no more than θ . Subsequently, the edge count of the transformed graph is perturbed by the Laplace mechanism with a sensitivity value of less than n . However, the choice of θ is critical for accurate estimation. A small θ reduces Laplace noise due to lower sensitivity, but results in significant edge loss during projection. Conversely, a θ close to n preserves more edges but increases the Laplace noise. Prior work addresses this balance using the exponential mechanism (EM) to prefer a θ that minimizes the combined errors arising from graph projection and the Laplace noise.

We outline this general approach in Algorithm 1. This algorithm takes in the dataset D , the constraint set Σ , a candidate set Θ for degree bounds, and privacy budgets ϵ_1 and ϵ_2 . These privacy budgets are later composed to get a final guarantee of ϵ -DP. We start by constructing the conflict graph $\mathcal{G}_{\Sigma}^D$ generated from the input dataset D and constraint set Σ (line 1), as defined in Section 2.1. Next, we sample in a DP manner a value of θ^* from the candidate set Θ with the privacy budget ϵ_1 (line 2). A baseline choice is an exponential mechanism detailed in Algorithm 2 to output a degree that minimizes the edge loss in a graph and the Laplace noise. In line 3, we compute a bounded graph $\mathcal{G}_{\theta^*}$ using the edge addition algorithm [12], we compute a θ^* -bounded graph $\mathcal{G}_{\theta^*}$ (detailed in Section 2). Finally, we perturb the true measure (either the number of edges for $\mathcal{I}_{\text{MI}}$ or the number of positive degree nodes for $\mathcal{I}_{\text{P}}$) on the projected graph, denoted by $f(\mathcal{G}_{\theta^*})$, by adding Laplace noise using the other privacy budget ϵ_2 (line 4).

The returned noisy measure at the last step has two sources of errors: (i) the bias incurred in the projected graph, i.e., $f(\mathcal{G}) - f(\mathcal{G}_{\theta^*})$, and (ii) the noise from the Laplace mechanism with an expected square root error $\sqrt{2\theta^*}/\epsilon_2$. Both errors depend on the selected parameter θ^* , and it is vital to select an optimal θ^* that minimizes the combined errors. Next, we describe a DP mechanism that helps select this parameter.

EM-based first try for parameter selection. The EM (Definition 5) specifies a quality function $q(\cdot, \cdot)$ that maps a pair of a database D and a candidate degree θ to a numerical value. The optimal θ value for a given database D should have the largest possible quality value and, hence, the highest probability of being sampled. We also denote $\theta_{\max}$ the largest degree candidate in Θ and use it as part of the quality function to limit its sensitivity.

The quality function we choose to compute the inconsistency measures includes two terms: for each $\theta \in \Theta$,

$$q_{\varepsilon_2}(\mathcal{G}, \theta) = -e_{\text{bias}}(\mathcal{G}, \theta) - \sqrt{2}\theta/\varepsilon_2 \quad (2)$$

where the first term e_{bias} captures the bias in the projected graph, and the second term $\sqrt{2}\theta/\varepsilon_2$ captures the error from the Laplace noise at budget ε_2 . For the minimum inconsistency measure $\mathcal{I}_{\text{MI}}$, we define the bias term as

$$e_{\text{bias}}(\mathcal{G}, \theta) = |\mathcal{G}_{\theta_{\max}} \cdot E| - |\mathcal{G}_{\theta} \cdot E| \quad (3)$$

i.e., the number of edges truncated at degree θ as compared to that at degree $\theta_{\max}$. For the problematic measure $\mathcal{I}_{\text{P}}$, we have

$$e_{\text{bias}}(\mathcal{G}, \theta) = |\mathcal{G}_{\theta_{\max}} \cdot V_{>0}| - |\mathcal{G}_{\theta} \cdot V_{>0}| \quad (4)$$

where $\mathcal{G}_{\theta} \cdot V_{>0}$ denote the nodes with positive degrees.

EXAMPLE 2. Consider the same graph as Example 1 and a candidate set $\Theta = [1, 2, 3]$ to compute the $\mathcal{I}_{\text{MI}}$ measure (number of edges) with $\varepsilon_2 = 1$. For the first candidate $\theta = 1$, as node 4 has degree 3, the edge addition algorithm would truncate 2 edges, for $\theta = 2$, 1 edge would be truncated and for $\theta = 3$, no edges would be truncated. We can, therefore, compute each term of the quality function for each θ given in Table 3. For this example, we see that $\theta = 1$ has the best quality even if it truncates the most number of edges as the error from Laplace noise overwhelms the bias error.

We summarize the basic EM for the selection of the bounded degree in Algorithm 2. This algorithm has a complexity of $O(|\Theta|m)$, where m is the edge size of the graph, as computing the quality function for each θ candidate requires running the edge addition algorithm once. The overall Algorithm 1 has a complexity of $O(|\Sigma|n^2 + |\Theta|m)$, where the first term is due to the construction of the graph.

Privacy analysis. The privacy guarantee of Algorithm 1 depends on the budget spent for the exponential mechanism and the Laplace mechanism, as summarized below.

Algorithm 2: EM-based first try for parameter selection

Data: Graph $\mathcal{G}$, candidate set Θ , quality function q , privacy budget $\varepsilon_1, \varepsilon_2$

Result: Candidate θ^*

- 1 Find the maximum value in Θ as $\theta_{\max}$
 - 2 For each $\theta_i \in \Theta$, compute $q_{\varepsilon_2}(\mathcal{G}, \theta_i)$ // See Equation (2)
 - 3 Sample θ^* with prob $\propto \exp(\frac{\varepsilon_1 q_{\varepsilon_2}(\mathcal{G}, \theta_i)}{2\theta_{\max}})$
 - 4 **Return** θ^*
-

Table 3. Quality function computation for $\mathcal{I}_{\text{MI}}$ for the conflict graph in Figure 1 when $\varepsilon_2 = 1$

θ	e_{bias}	$\sqrt{2}\theta/\varepsilon_2$	q
1	2	$\sqrt{2}$	$-2 - \sqrt{2}$
2	1	$2\sqrt{2}$	$-1 - 2\sqrt{2}$
3	0	$3\sqrt{2}$	$-3\sqrt{2}$

THEOREM 1. *Algorithm 1 satisfies $(\varepsilon_1 + \varepsilon_2)$ -node DP for $\mathcal{G}_\Sigma^D$ and $(\varepsilon_1 + \varepsilon_2)$ -DP for the input database D .*

PROOF SKETCH. The proof is based on the sequential composition of two DP mechanisms as stated in Proposition 1. $\square$

As stated below, we just need to analyze the sensitivity of the quality function in the exponential mechanism and the sensitivity of the measure over the projected graph.

LEMMA 1. *The sensitivity of $f \circ \pi_\theta(\cdot)$ in Algorithm 1 is θ , where π_θ is the edge addition algorithm with the input θ and $f(\cdot)$ counts edges for $\mathcal{I}_{MI}$ and nodes with a positive degrees for $\mathcal{I}_P$.*

PROOF SKETCH. For $\mathcal{I}_P$, we can analyze a worst-case scenario where the graph is a star with n nodes such that there is an internal node connected to all other $n - 1$ nodes, and the threshold θ for edge addition is n . The edge addition algorithm would play a minimal role, and no edges would be truncated. For a neighboring graph that differs on the internal node, all edges of the graph are removed (connected to the internal node), and the $\mathcal{I}_P = 0$ (no problematic nodes), making the sensitivity for $\mathcal{I}_P$ in this worst-case $= n$.

For $\mathcal{I}_{MI}$, the proof is similar to prior work [12] for publishing degree distribution that uses stable ordering to keep track of the edges for two neighboring graphs. We need to analyze the changes made to the degree of each node by adding one edge at a time for two graphs $\mathcal{G}$ and its neighboring graph $\mathcal{G}'$ with an additional node v^+ . The graphs have the stable ordering of edges (Definition 7) Λ and Λ' , respectively. Assuming the edge addition algorithm adds a set of t extra edges incident to v^+ for $\mathcal{G}'$, we can create t intermediate graphs and their respective stable ordering of edges that can be obtained by removing from the stable ordering Λ' each edge t and others that come after t in the same sequence as they occur in Λ' . We analyze consecutive intermediate graphs, their stable orderings, and the edges actually that end up being added by the edge addition algorithm. As the edge addition algorithm removes all edges of a node once an edge incident is added, we observe that only one of these t edges is added. All other edges incident to v^+ are removed. We prove this extra edge leads to decisions in the edge addition algorithm that always restricts such consecutive intermediate graphs to differ by at most 1 edge. This proves the lemma for $\mathcal{I}_{MI}$ as at most t (upper bounded by θ) edges can differ between two neighboring graphs. $\square$

The full proof for the above lemma can be found in the full paper [47]. We now analyze the sensitivity of the quality function using both measures' sensitivity analysis.

LEMMA 2. *The sensitivity of the quality function $q_{\varepsilon_2}(\mathcal{G}, \theta_i)$ in Algorithm 2 defined in Equation (2) is $\theta_{\max} = \max(\Theta)$.*

PROOF SKETCH. We prove the theorem for the $\mathcal{I}_{MI}$ measure and show that it is similar for $\mathcal{I}_P$. The sensitivity of the quality function is computed by comparing the respective quality functions of two neighboring graphs $\mathcal{G}$ and $\mathcal{G}'$ with an extra node. It is upper bound by the difference of two terms $(|\mathcal{G}'_{\theta_{\max}}.E| - |\mathcal{G}_{\theta_{\max}}.E|) - (|\mathcal{G}'_{\theta}.E| - |\mathcal{G}_{\theta}.E|)$. The first term $(|\mathcal{G}'_{\theta_{\max}}.E| - |\mathcal{G}_{\theta_{\max}}.E|)$ is the sensitivity of the measures, as already proved by Lemma 1 is equal to $\theta_{\max}$. The second term $(|\mathcal{G}'_{\theta}.E| - |\mathcal{G}_{\theta}.E|)$ is always ≥ 0 as $|\mathcal{G}'_{\theta}.E| \geq |\mathcal{G}_{\theta}.E|$ as discussed in the proof for Lemma 1. $\square$

Utility analysis. The utility of Algorithm 1 is directly encoded by the quality function of the exponential mechanism in Algorithm 2. We first define the best possible quality function value for a given database and its respective graph as

$$q_{\text{opt}}(D, \varepsilon_2) = \max_{\theta \in \Theta} q_{\varepsilon_2}(\mathcal{G}_\Sigma^D, \theta) \quad (5)$$

and the set of degree values that obtain the optimal quality value as

$$\Theta_{\text{opt}} = \{\theta \in \Theta : q_{\varepsilon_2}(\mathcal{G}_{\Sigma}^D, \theta) = q_{\text{opt}}(D, \varepsilon_2)\}. \quad (6)$$

However, we define e_{bias} as the difference in the number of edges or nodes in the projected graph $\mathcal{G}_{\theta}$ compared to that of $\mathcal{G}_{\theta_{\text{max}}}$, instead of $\mathcal{G}$. This is to limit the sensitivity of the quality function. To compute the utility, we slightly modify the quality function without affecting the output of the exponential mechanism.

$$\tilde{q}_{\varepsilon_2}(\mathcal{G}, \theta) = q_{\varepsilon_2}(\mathcal{G}, \theta) + f(\mathcal{G}_{\theta_{\text{max}}}) - f(\mathcal{G}_{\Sigma}^D), \quad (7)$$

where $f(\cdot)$ returns edge count for $\mathcal{I}_{\text{MI}}$ and the number of nodes with positive degrees for $\mathcal{I}_{\text{P}}$. This modified quality function should give the same set of degrees Θ_{opt} with optimal values equal to

$$\tilde{q}_{\text{opt}}(D, \varepsilon_2) = \max_{\theta \in \Theta} q_{\varepsilon_2}(\mathcal{G}_{\Sigma}^D, \theta) + f(\mathcal{G}_{\Sigma \theta_{\text{max}}}^D) - f(\mathcal{G}_{\Sigma}^D). \quad (8)$$

Then, we derive the utility bound for Algorithm 1 based on the property of the exponential mechanism as follows.

THEOREM 2. *On any database instance D and its respective conflict graph $\mathcal{G}_{\Sigma}^D$, let o be the output of Algorithm 1 with Algorithm 2 over D . Then, with a probability of at least $1 - \beta$, we have*

$$|o - a| \leq -\tilde{q}_{\text{opt}}(D, \varepsilon_2) + \frac{2\theta_{\text{max}}}{\varepsilon_1} (\ln \frac{2|\Theta|}{|\Theta_{\text{opt}}| \cdot \beta}) \quad (9)$$

where a is the true inconsistency measure over D and $\beta \leq \frac{1}{e^{\sqrt{2}}}$.

PROOF SKETCH. We use the probabilistic utility bound of the exponential mechanism [46] that guarantees that a suitable candidate is sampled with probability $1 - \beta$ for a given quality function. To prove the bound, we utilize the optimal quality function $\tilde{q}_{\text{opt}}(D)$ and the error from the Laplace mechanism with the exponential mechanism's utility bound. The full proof is in the full version [47]. $\square$

This theorem indicates that the error incurred by Algorithm 1 with Algorithm 2 is directly proportional to the log of the candidate size $|\Theta|$ and the sensitivity of the quality function. The β parameter in the theorem is a controllable probability parameter. According to the accuracy requirements of a user's analysis, one may set β as any value less than this upper bound. For example, if we set $\beta = 0.01$, then our theoretical analysis of Algorithm 2 that says the algorithm's output being close to the true answer will hold with a probability of $1 - \beta = 0.99$. We also show a plot to show the trend of the utility analysis as a function of β in Appendix A.5 [47]. Without prior knowledge about the graph, θ_{max} is usually set as the number of nodes n , and Θ includes all possible degree values up to n , resulting in poor utility. Fortunately, for our use case, the edges in the graph arise from the DCs that are available to us. In the next section, we show how we can leverage these constraints to improve the utility of our algorithm by truncating candidates in the set Θ .

4.2 Optimized Parameter Selection

Our developed strategy to improve the parameter selection includes two optimization techniques. The overarching idea behind these optimizations is to gradually truncate large candidates from the candidate set Θ based on the density of the graph. For example, we observe that the Stock dataset [49] has a sparse conflict graph, and its optimum degree for graph projection is in the range of $10^0 - 10^1$. In contrast, the graph for the Adult dataset sample [3] is extraordinarily dense and has an optimum degree θ greater than 10^3 , close to the sampled data size. Removing unneeded large

candidates, especially those greater than the true maximum degree of the graph, can help the high sensitivity issue of the quality function and improve our chances of choosing a better bound.

Our first optimization estimates an upper bound for the true maximum degree of the conflict graph and removes candidates larger than this upper bound from the initial candidate set. The second optimization is a hierarchical exponential mechanism that utilizes two steps of exponential mechanisms. The first output, θ^1 , is used to truncate Θ further by removing candidates larger than θ^1 from the set, and the second output is chosen as the final candidate θ^* . In the rest of this section, we dive deeper into the details of these optimizations and discuss their privacy analysis.

Estimating the degree upper bound using FDs. Given a conflict graph $\mathcal{G}(V, E)$, we use $d(\mathcal{G}, v)$ to denote the degree of the node $v \in V$ in $\mathcal{G}$ and $d_{\max}(\mathcal{G}) = \max_{v \in V} d(\mathcal{G}, v)$ to denote the maximum degree in $\mathcal{G}$. We estimate $d_{\max}$ by leveraging how conflicts were formed for its corresponding dataset D under Σ .

The degree for each vertex in $\mathcal{G}$ can be found by going through each tuple t in the database D and counting the tuples that violate the Σ jointly with t . However, computing this value for each tuple is computationally expensive and highly sensitive, making it impossible to learn directly with differential privacy. We observe that the conflicts that arise due to functionality dependencies (FDs) depend on the values of the left attributes in the FD.

EXAMPLE 3. Consider the same setup as Example 1 and an FD $\sigma : \text{Capital} \rightarrow \text{Country}$. We can see that the number of violations added due to the erroneous grey row is 3. This number is also one smaller than the maximum frequency of values occurring in the Capital attribute, and the most frequent value is “Ottawa”.

Based on this observation, we can derive an upper bound for the maximum degree of a conflict graph if it involves only FDs, and this upper bound has a lower sensitivity. We show the upper bound in Lemma 3 for one FD first and later extend for multiple FDs.

LEMMA 3. Given a database D and a FD $\sigma : X \rightarrow Y$ as the single constraint, where $X = \{A_1, \dots, A_k\}$ and Y is a single attribute. For its respective conflict graph $\mathcal{G}_{\Sigma=\{\sigma\}}^D$, simplified as $\mathcal{G}_{\sigma}^D$, we have the maximum degree of the graph $d_{\max}(\mathcal{G}_{\sigma}^D)$ upper bounded by

$$d_{\text{bound}}(D, X) = \max_{\vec{a}_X \in \text{dom}(A_1) \times \dots \times \text{dom}(A_k)} \text{freq}(D, \vec{a}_X) - 1, \quad (10)$$

where $\text{freq}(D, \vec{a}_X)$ is the frequency of values $\vec{a}_X$ occurring for the attributes X in the database D . The sensitivity for $d_{\text{bound}}(D, X)$ is 1.

PROOF. An FD violation can only happen to a tuple t with other tuples t' that share the same values for the attributes X . Let $\vec{a}_X^*$ be the most frequent value for X in D , i.e.,

$$\vec{a}_X^* = \text{argmax}_{\vec{a}_X \in \text{dom}(A_1) \times \dots \times \text{dom}(A_k)} \text{freq}(D, \vec{a}_X).$$

In the worst case, a tuple t has the most frequent value $\vec{a}_X^*$ for X but has a different value in Y with all the other tuples with $X = \vec{a}_X^*$. Then the number of violations involved by t is $\text{freq}(D, \vec{a}_X^*) - 1$.

Adding a tuple or removing a tuple to a database will change, at most, one of the frequency values by 1. Hence, the sensitivity of the maximum frequency values is 1. $\square$

Now, we will extend the analysis to multiple FDs.

THEOREM 3. Given a database D and a set of FDs $\Sigma = \{\sigma_1, \dots, \sigma_l\}$, for its respective conflict graph $\mathcal{G}_{\Sigma}^D$, we have the maximum degree of the graph $d_{\max}(\mathcal{G}_{\Sigma}^D)$ upper bounded by

$$d_{\text{bound}}(D, \Sigma) = \sum_{(\sigma: X \rightarrow Y) \in \Sigma} d_{\text{bound}}(D, X) \quad (11)$$

PROOF. By Lemma 3, for each FD $\sigma : X \rightarrow Y$, a tuple may violate at most $d_{\text{bound}}(D, X)$ number of tuples. In the worst case, the same tuple may violate all FDs. $\square$

We will spend some privacy budget ϵ_0 to perturb the upper bound $d_{\text{bound}}(D, X)$ for all FDs with LM and add them together. Each FD is assigned with $\epsilon_0/|\Sigma_{\text{FD}}|$, where Σ_{FD} is the set of FDs in Σ . We denote this perturbed upper bound as $\tilde{d}_{\text{bound}}$ and add it to the candidate set Θ if absent.

Extension to general DCs. The upper bound derived in Theorem 3 only works for FDs but fails for general DCs. General DCs have more complex operators, such as “greater/smaller than,” in their formulas. Such inequalities require the computation of tuple-specific information, which is hard with DP. For example, consider the DC $\sigma : \neg(t_i[\text{gain}] > t_j[\text{gain}] \wedge t_i[\text{loss}] < t_j[\text{loss}])$ saying that if the gain for tuple t_i is greater than the gain for tuple t_j , then the loss for t_i should also be greater than t_j . We can observe that similar analyses for FDs do not work here as the frequency of a particular domain value in D does not bound the number of conflicts related to a tuple. Instead, we have to iterate each tuple t 's gain value and find how many other tuples t' s violate this gain value. In the worst case, such a computation may have a sensitivity equal to the data size. Therefore, estimation using DCs may result in much noise, especially when the dataset has fewer conflicts, and the noise is added to correspond to the large sensitivity.

Our experimental study (Section 6) shows that datasets with general DCs have dense conflict graphs, which favors larger θ s for graph projection. Hence, if we learn a small noisy upper bound $\tilde{d}_{\text{bound}}$ based on the FDs with LM, we will first prune all degree candidates smaller than $\tilde{d}_{\text{bound}}$, but then include $|V|$, which corresponds to the case when no edges are truncated, and Laplace mechanism is applied with the largest possible sensitivity $|V|$, i.e.,

$$\Theta' = \{\theta \in \Theta \mid \theta \leq \tilde{d}_{\text{bound}}\} \cup \{|V|\}. \quad (12)$$

Though the maximum value in Θ' is $|V|$, the sensitivity of the quality function over the candidate set Θ' remains $\tilde{d}_{\text{bound}}$. For the $|V|$ candidate, the quality function only depends on the Laplace error $\frac{\sqrt{2}|V|}{\epsilon_2}$ and has no error from ϵ_{bias} as no edges will be truncated. Despite being tailored for FDs, we show that, in practice, our approach is cheap and performs well for DCs. In Section 6, we show that this approach works well for the dense Adult [38] dataset where we compute the $\mathcal{I}_P$ using this strategy in Figure 5. Developing a specific strategy for DCs is an important direction of future work.

In practice, one may skip this upper bound calculation process and skip directly to the two-step exponential mechanism if it is known that the graph is too dense or contains few FDs and more general DCs. We discuss this in detail in the experiments section.

Hierarchical EM. The upper bound d_{bound} may not be tight as it estimates the maximum degree in the worst case. The graph would be sparse with low degree values, and there is still room for pruning. To further prune candidate values in the set Θ , we use a hierarchical EM that first samples a degree value θ^* to prune values in Θ and then sample again another value θ^* from the remaining candidates as the final degree the graph projection. Our work uses a two-step hierarchical EM by splitting the privacy budget equally into halves. One may extend this EM to more steps at the cost of breaking their privacy budget more times, but in practice, we notice that a two-step is enough for a reasonable estimate.

EXAMPLE 4. Consider the same setup as Example 1. For this dataset, we start with $\Theta = [1, 2, 3]$ and the θ_{max} for this setup is 3. Assume no values are pruned in the first optimization phase. We compare a single versus a two-step hierarchical EM for the second optimization step. From Table 4 in Example 2, we know that the θ_1 has the best quality. However, as the quality values are close, the probability of choosing the best candidate is similar, as shown in Table 4 with $\epsilon = 1$. The exponential mechanism

Table 4. Probabilities of candidates with the exponential mechanism (EM) vs. the two-step hierarchical exponential mechanism (2-EM). θ_1^* refers to the first-step output of 2-EM.

θ	q	EM	2-EM ($\theta_1^* = \theta_3$)	2-EM ($\theta_1^* = \theta_2$)
1	-3.41	0.35	0.51	1
2	-3.82	0.33	0.49	-
3	-4.24	0.31	-	-

Algorithm 3: Optimized EM for parameter selection

Data: Graph $\mathcal{G}(V, E)$, candidate set $\Theta = \{1, \dots, |V|\}$, quality function q , privacy budget $\varepsilon_1, \varepsilon_2$

Result: Candidate θ^*

```

1 if  $\Sigma$  mainly consists of FDs then
2    $\varepsilon_0 \leftarrow \varepsilon_1/4, \varepsilon_1 \leftarrow \varepsilon_1 - \varepsilon_0$ 
3   Compute noisy upper bound  $\tilde{d}_{\text{bound}} \leftarrow \sum_{\sigma: X \rightarrow Y} (d_{\text{bound}}(D, X) + \text{Lap}(|\Sigma_{\text{FD}}|/\varepsilon_0))$ 
4   Prune candidates  $\Theta \leftarrow \{\theta \in \Theta \mid \theta \leq \tilde{d}_{\text{bound}}\} \cup \{\tilde{d}_{\text{bound}}, |V|\}$ 
5   Set  $\theta_{\text{max}} \leftarrow \min(\tilde{d}_{\text{bound}}, |V|)$ 
6   for  $s \in \{1, 2\}$  do
7     For each  $\theta_i \in \Theta$ , compute  $q_{\varepsilon_2}(\mathcal{G}, \theta_i)$  // See Equation (2)
8     Sample  $\theta^*$  with prob  $\propto \exp(\frac{\varepsilon_1}{2} \frac{q_{\varepsilon_2}(\mathcal{G}, \theta_i)}{\theta_{\text{max}}})$ 
9     Prune candidates  $\Theta \leftarrow \{\theta \in \Theta \mid \theta \leq \theta^*\}$ 
10    Set  $\theta_{\text{max}} \leftarrow \theta^*$ 
11 Return  $\theta^*$ 

```

will likely choose a suboptimal candidate in such a scenario as the probabilities are close. But if a two-step exponential mechanism is used even with half budget $\varepsilon = 0.5$, the likelihood of choosing the best candidate θ_1 goes up to 0.51 if the first step chose θ_3 or 1 if the first step chosen θ_2 .

Incorporating the optimizations into the algorithm. Algorithm 3 outlines the two optimization techniques. First, we decide when to use the estimated upper bound for the maximum degrees, for example, when the constraint set Σ mainly consists of FDs. We will spend part of the budget ε_0 from ε_1 to perturb the upper bounds $d_{\text{bound}}(D, X)$ for all FDs with Laplace mechanism and add them together (lines 1-3). The noisy upper bound $\tilde{d}_{\text{bound}}$ prunes the candidate set (line 4). We also add $|V|$ to the candidate set if there are general DCs in Σ , and then set the sensitivity of the quality function θ_{max} to be the minimum of the noisy upper bound or $|V|$ (line 5). Then, we conduct the two-step hierarchical exponential mechanism for parameter selection (lines 6-10). Lines 7-8 work similarly to the previous exponential mechanism algorithm with half of the remaining ε_1 , where we choose a θ^* based on the quality function. However, instead of using it as the final candidate, we use it to prune values in Θ and improve the sensitivity θ_{max} for the second exponential mechanism (lines 9-10). Then, we repeat the exponential mechanism and output the sampled θ^* (line 11). Algorithm 3 has a similar complexity of $O(|\Theta|m)$ as Algorithm 2, where $|E|$ is the edge size of the graph. The overall Algorithm 1 has a complexity of $O(|\Sigma|n^2 + |\Theta|m)$.

Privacy and utility analysis. The privacy analysis of the optimizations depends on the analysis of three major steps: d_{bound} computation with the Laplace mechanism, the two-step exponential mechanism, and the final measure calculation with the Laplace mechanism. By sequential composition, we have Theorem 4.

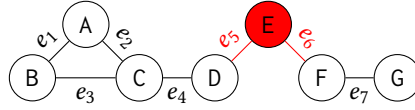


Fig. 2. Toy graph example $\mathcal{G}$ with seven nodes (A to G) and seven edges. Consider a neighboring graph $\mathcal{G}'$ with the differing node E (red) and its two edges.

THEOREM 4. *Algorithm 1 with the optimized EM in Algorithm 3 satisfies $(\epsilon_1 + \epsilon_2)$ -DP.*

PROOF SKETCH. The proof is similar to Theorem 1 and is due to the composition property of DP as stated in Proposition 1. $\square$

We show a tighter sensitivity analysis for the quality function in EM over the pruned candidate set. The sensitivity analysis is given by Lemma 4 and is used for $\theta_{\max}$ in line 8 of Algorithm 3.

LEMMA 4. *The sensitivity of $q_{e_2}(\mathcal{G}, \theta_1)$ in the 2-step EM (Algorithm 3) defined in Equation (2) is $\theta_{\max} = \min(\tilde{d}_{\text{bound}}, |V|)$ for 1st EM step and $\theta_{\max} = \theta^*$ for the 2nd EM step.*

PROOF SKETCH. The proof follows from Lemma 2, substituting the $\theta_{\max}$ with the appropriate threshold values for each EM step. $\square$

The utility analysis in Theorem 2 for Algorithm 1 with the basic EM (Algorithm 2) still applies to the optimized EM (Algorithm 3). The basic EM usually has $\theta_{\max} = |D| = |V|$ and the full budget ϵ_1 , while the optimized EM has a much smaller $\theta_{\max}$ and slightly lower privacy budget when the graph is sparse. In practice (Section 6), we see significant utility improvements by the optimized EM for sparse graphs. When the graph is dense, we see the utility degrade slightly due to a smaller budget for each EM. However, the degradation is negligible with respect to the true inconsistency measure.

5 DP Minimum Vertex Cover for $\mathcal{I}_R$

This section details our approach for computing the optimal repair measure, $\mathcal{I}_R$, using the conflict graph. $\mathcal{I}_R$ is defined as the minimum number of vertices that must be removed to eliminate all conflicts within the dataset. For the conflict graph $\mathcal{G}_S^D$, this corresponds to finding the minimum vertex cover – an NP-hard problem. To address this, we apply a well-known polynomial-time algorithm that provides a 2-approximation for vertex cover [61]. This randomized algorithm iterates through a random ordering of edges, adding both nodes of each edge to the vertex cover if they haven't been encountered, then removes all incident edges. The process repeats until the edge list is exhausted. In our setting, we aim to compute the minimum vertex cover size while satisfying DP. A straightforward approach would be to analyze the sensitivity of the 2-approximation algorithm and add the appropriate DP noise. However, determining the sensitivity of this naive approximation is challenging, as the algorithm's output can fluctuate significantly depending on the order of selected edges. This variability is illustrated in Example 5.

EXAMPLE 5. *Let us consider a graph $\mathcal{G}$ with 7 vertices A to G and 7 edges e_1 to e_7 as shown in Figure 2. We can have a neighboring graph $\mathcal{G}'$ by considering the vertex E as the differing vertex and two of its edges e_5 and e_6 as the differing edges. This example shows that according to the vanilla 2-approximate algorithm, the output for the graphs $\mathcal{G}$ and $\mathcal{G}'$ may vary drastically. For $\mathcal{G}$, if e_2 is selected followed by e_7 , then the vertex cover size is 4. However, for graph $\mathcal{G}'$, if e_1 or e_4 is selected first and subsequently after the other one e_6 is selected, then the output is 6. Moreover, this difference may get significantly large if the above graph is stacked multiple times and the corresponding vertex that creates this difference is chosen every time.*

Algorithm 4: DP approximation of minimum vertex cover size for $\mathcal{I}_R$ **Data:** Graph $\mathcal{G}(V, E)$, stable global ordering Λ , privacy parameter ε **Result:** DP minimum vertex cover size for $\mathcal{I}_R$

```

1 Initialize vertex cover set  $C = \emptyset$  and size  $c = 0$ 
2 Initialize edge list  $E_0 = E$ 
3 for  $i \in \{1 \dots |\Lambda|\}$  do
4   pop edge  $e_i = \{u, v\}$  in order from  $\Lambda$ 
5   add  $u$  and  $v$  to  $C$  and  $c = c + 2$ 
6    $E_{i+1} =$  remove all edges incident to  $u$  or  $v$  from  $E_i$ 
7 Return  $c + \text{Lap}(\varepsilon/2)$ 

```

To solve the sensitivity issue, we make a minor change in the algorithm by traversing the edges in a particular order (drawing on [12]). We use a similar stable ordering Λ defined in Section 2.3. The new algorithm is shown in Algorithm 4. We initialize an empty vertex cover set C , its size c , and an edge list (lines 1–2). We then start an iteration over all edges in the same ordering as the stable ordering Λ (line 3). For each edge $e_i = \{u, v\} \in E$ that is part of the graph, we add both u and v to C and correspondingly increment the size c (lines 4–5). We remove all other edges, including e_i , connected to u or v from E and continue the iteration (line 5). Finally, we return the noisy size of the vertex cover (line 6). The sensitivity of this algorithm is given by Proposition 3.

PROPOSITION 3. *Algorithm 4 obtains a vertex cover, and its size has a sensitivity of 2.*

PROOF. (sketch) Consider two graphs that differ by one node v^* and the edges connected to it. The stable ordering of edges Λ in algorithm 4 restricts the order in which the edges occur in both graphs. As the algorithm removes all edges of a particular node once observed, we can delineate 3 cases depending on which of the two graphs has the differing node. This allows us to show proof by induction. The detailed proof is in the full version [47]. $\square$

EXAMPLE 6. *Let us consider our running example in Figure 2 as input to Algorithm 4 and use it to understand the proof. We have two graphs – $\mathcal{G}$ which has 6 vertices $V = [A, B, C, D, F, G]$ and edges $E = [e_1, e_2, e_3, e_4, e_7]$ and $\mathcal{G}'$ has 7 vertices $V' = [A, B, C, D, E, F, G]$ and edges $E' = [e_1, e_2, \dots, e_7]$. The total possible number of edges is $\binom{7}{2} = 21$, and we can have a global stable ordering of the edges Λ depending on the lexicographical ordering of the vertices as $e_1, e_2, e_3, \dots, e_{21}$. When the algorithm starts, both vertex cover sizes are initialized to $c = 0, c' = 0$, and the algorithm's state is in Case 1 with $v^* = E$. We delineate the next steps of the algorithm below:*

- *Iteration 1 (Subcase 1b) : $e_1(A, B)$ is chosen. A and B are both in E_0 and E'_0 . Hence $c = 2, c' = 2$.*
- *Iteration 2, 3 (Subcase 1c) : $e_2(A, C)$ and $e_3(B, C)$ are chosen. Both are removed in iteration 1. Hence $c = 2, c' = 2$.*
- *Iteration 4 (Subcase 1b) : $e_4(C, D)$ is chosen. C and D are both in E_3 and E'_3 . Hence $c = 4, c' = 4$.*
- *Iteration 5 (Subcase 1c) : $e_5(D, E)$ is chosen, removed from E'_4 in Iteration 4, and was never present in E . Hence $c = 4, c' = 4$.*
- *Iteration 6 (Subcase 1a) : $e_6(E, F)$ is chosen. It is in E'_5 but not in E_5 . Hence, $c = 4, c' = 6$ and the new $v^* = E$.*
- *Iteration 7 (Subcase 2a) : $e_7(F, G)$ is chosen. It is in E_6 but removed from E_6 in Iteration 6. Hence, $c = 6, c' = 6$, and the algorithm is complete.*

Privacy and utility analysis. We now show the privacy and utility analysis of Algorithm 4 using Theorem 5 below.

THEOREM 5. *Algorithm 4 satisfies ϵ -node DP and, prior to adding noise in line 7, obtains a 2-approximation vertex cover size.*

PROOF. The Algorithm 4 satisfies ϵ -node DP as we calculate the private vertex cover using the Laplace mechanism with sensitivity 2 according to Proposition 3. It is also a 2-approximation as the stable ordering Λ in Algorithm 4 can be perceived as one particular random order of the edges and hence has the same utility as the original 2-approximation algorithm. $\square$

6 Experiments

This section presents our experiment results on computing the three measures outlined in Section 4 and Section 5. The questions that we ask through our experiments are as follows:

- (1) How far are the private measures from the true measures?
- (2) How do the different strategies for the degree truncation bound compare against each other?
- (3) How do our methods perform at different privacy budgets?

6.1 Experimental Setup

All our experiments are performed on a server with Intel Xeon Platinum 8358 CPUs (2.60GHz) and 1 TB RAM. Our code is in Python 3.11 and can be found in the artifact submission. All experiments are repeated for 10 runs, and the mean error value is reported.

Datasets and violation generation. We replicate the exact setup as Livshits et al. [40] for experimentation. We conduct experiments on five real-life datasets and their corresponding DCs as described in Table 5. These datasets are initially consistent with the constraints. All experiments are done on a subset of 10k rows, and violations are added similarly using both their proposed algorithms, namely CONoise (for Constraint-Oriented Noise) and RNoise (for Random Noise). CONoise, in every iteration, randomly selects a constraint and two tuples and changes the values of the attributes in those tuples participating in the constraint. In our experiments, we run 200 iterations of the CONoise violations. Similarly, RNoise parameterized by α selects α values in the dataset and changes its value to another value from the active domain of the corresponding attribute (with probability 0.5) or to a typo. The datasets vary immensely in the density of their conflict graphs as described in the max degree column of Table 5. For example, the Adult 10k nodes subset has a maximum degree of 9635, whereas the Stock dataset has a maximum of 1 with the same amount of conflict addition.

Metrics. Following Livshits et al. [40], we randomly select a subset of 10k rows of each dataset, add violations to the subset, and compute the inconsistency measures on the dataset with violations. To measure performance, we utilize the normalized ℓ_1 distance error [17], $|I(D, \Sigma) - \mathcal{M}(D, \Sigma, \epsilon)|/I(D, \Sigma)$, where $\mathcal{M}(D, \Sigma, \epsilon)$ represents the estimated private value of the measure and $I(D, \Sigma)$ denotes the true value. For I_R , we use the linear approximation algorithm from Livshits et al. [40] to estimate the non-private value.

Table 5. Description of datasets. The max deg column shows the maximum degree of any node in a 10k rows subset of the conflict graph of the dataset with 1% RNoise.

Dataset	#Tuple	#Attrs	#DCs(#FDs)	Max Deg 1% RNoise
Adult [3]	32561	15	3 (2)	9635
Flight [48]	500000	20	13 (13)	1520
Hospital [53]	114919	15	7 (7)	793
Stock [49]	122498	7	1 (1)	1
Tax [10]	1000000	15	9 (7)	373

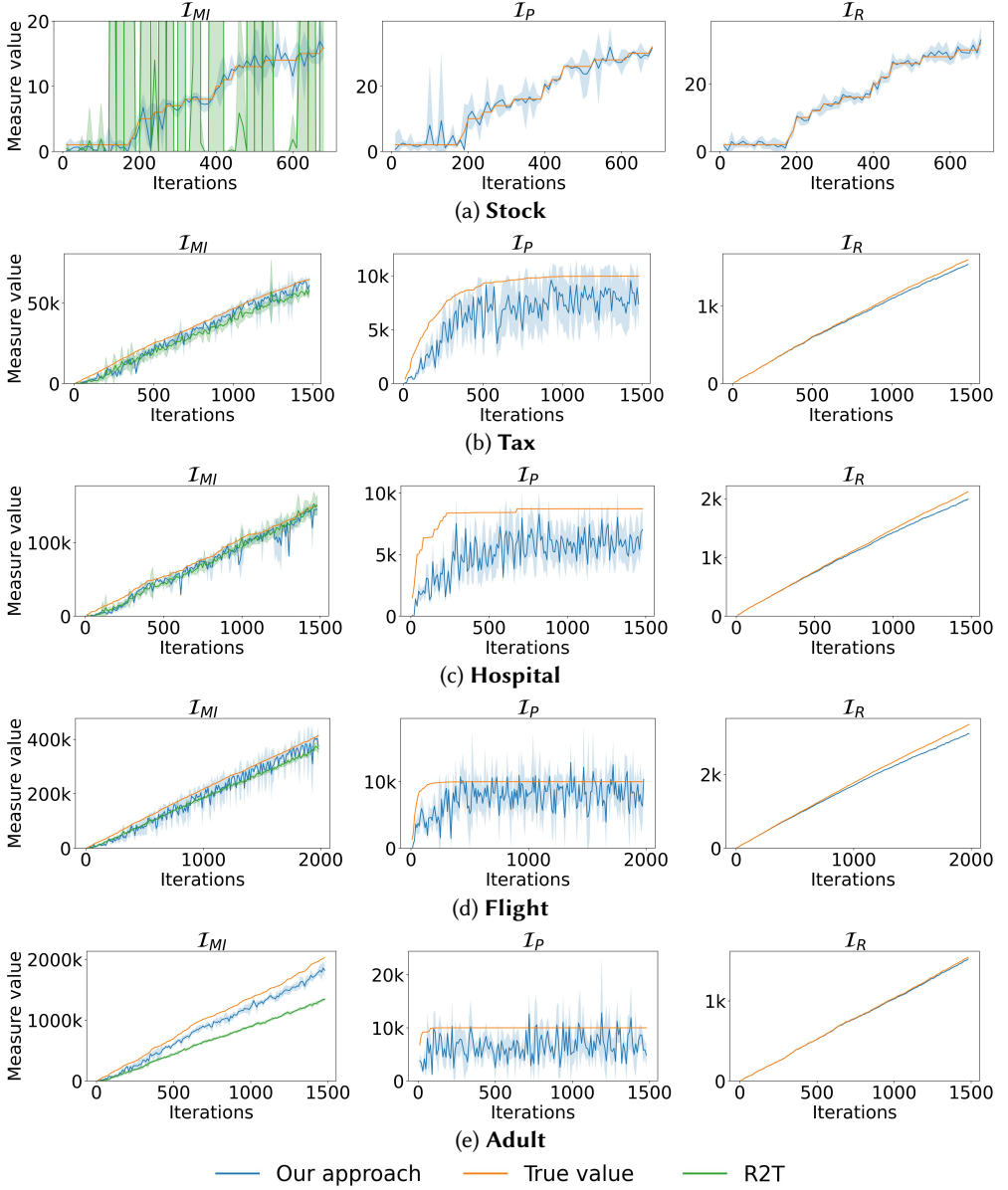


Fig. 3. True vs Private estimates for all datasets with RNoise $\alpha = 0.01$ at $\epsilon = 1$. Each row shows the three measures — I_P (Positive degree nodes), I_{MI} (Number of edges), and I_R (Size of vertex cover) — for the corresponding dataset.

Algorithm variations. We experiment with multiple different variations of Algorithm 1 for I_{MI} and I_P . The initial candidate set for the degree bound is $\Theta = [1, 5, 10, 100, 500, 1000, 2000, 3000, \dots, 10000]$ with multiples of 1000 along with some small candidates.

- *Baseline 1:* naively sets the bound θ^* to the maximum possible degree $|V|$ in Algorithm 1 by skipping line 2 and the unused privacy budget ϵ_1 is used for the final noise addition step.

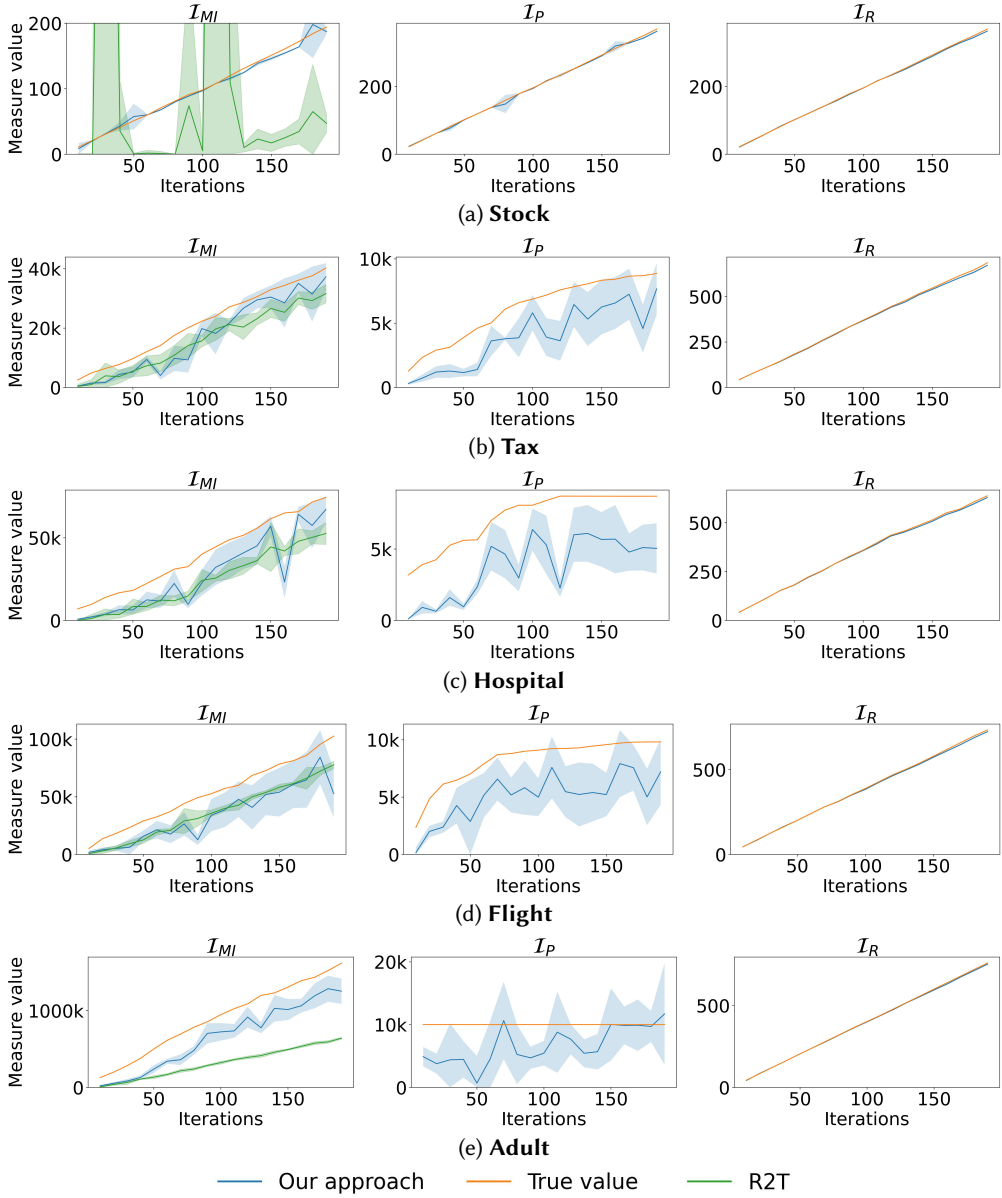


Fig. 4. True vs Private estimates for all datasets with RNoise $\alpha = 0.01$ at $\epsilon = 1$. Each row shows the three measures — I_{ML} (Number of edges), I_P (Positive degree nodes), and I_R (Size of vertex cover) — for the corresponding dataset.

- *Baseline 2*: sets the bound θ^* to the actual maximum degree of the conflict graph $d_{\max}(\mathcal{G}_{\Sigma}^D)$. Note that this is a non-private baseline that only acts as an upper bound and is one of the best values that can be achieved without privacy constraints.
- *Exponential mechanism*: choose θ^* over the complete candidate set Θ using the basic EM in Algorithm 2.

- *Hierarchical exponential mechanism*: chooses θ^* using a two-step EM with an equal budget for each step in Algorithm 3, but skipping Lines 1-5 of the upper bound computation step.
- *Upper bound + hierarchical exponential mechanism (our approach)*: encompasses both the optimization strategies, including the upper bound computation and the hierarchical exponential mechanism discussed in Section 4.2 (the full Algorithm 3).

By default, we experiment with a total privacy budget of $\epsilon = 1$ unless specified otherwise.

6.2 Results

True vs private estimation. In Figures 3 and 4, we plot the true vs. private estimates at $\epsilon = 1$ for all datasets with RNoise ($\alpha = 0.01$) and CONoise (200 iterations) respectively. The datasets are ordered according to their densities from top to bottom. Each figure contains the measured value (I_{MI} , I_P , or I_R) on the Y-axis and the number of iterations on the X-axis. For the CONoise, the number of iterations is set to 200 for every dataset, and for RNoise, the iterations correspond to the number of iterations required to reach 1% ($\alpha = 0.01$) number of random violations. The orange line corresponds to the true value of the measure, and the blue line corresponds to the private measure using our approach. For I_{MI} and I_P measures, the blue line represents the upper bound + hierarchical exponential mechanism strategy described in Section 4.2 along with its standard deviation in shaded blue. For the I_R measure, it represents the private minimum vertex cover size algorithm. We also add a baseline approach using a state-of-the-art private SQL approach called R2T [14]. We add this baseline only for the I_{MI} measure as I_R cannot be written with SQL, and I_P requires the DISTINCT/GROUP BY clause that R2T does not support. Based on the experiments, we draw three significant observations.

First, compared to the SQL baseline (R2T), our approach has a better relative error on average across all datasets. However, R2T is slightly behind for moderate to high dense datasets such as Tax (0.207 vs. 0.334), Hospital (0.209 vs. 0.386), and Flight (0.202 vs. 0.205) and Adult (0.187 vs. 0.269) but falls short for sparse datasets such as Stock (0.492 vs. 137.05). This is because the true value of the measure is small, and R2T adds large amounts of noise.

Second, our approach for the I_{MI} and I_P fluctuates more and has a higher standard deviation compared to the I_R measure. This is because of the privacy noise due to the relatively high sensitivity of our upper bound + hierarchical exponential mechanism approach. On the other hand, the vertex cover size approach for I_R has a sensitivity equal to 2 and, therefore, does not show much fluctuation when the true measure value is large enough.

Third, we observe that our approach generally performs well across all five datasets and all inconsistency measures. The I_{MI} and I_P measures have average errors of 0.25 and 0.46, respectively, across all datasets where Stock is the worst performing dataset for I_{MI} and Adult is the worst performing for I_P . The I_R performs the best with an average error of 0.08, with Stock as the worst-performing dataset. We investigate the performance of each dataset in detail in our next experiment and find out that the density of the graphs plays a significant role in the performance of our algorithms.

Comparing different strategies for choosing θ . In Figure 5, we present the performance of different algorithm variations in computing I_P and I_{MI} for all datasets using RNoise at $\alpha = 0.01$ and $\epsilon = 1$. The y-axis in each figure shows the logarithmic scaled error, while the x-axis displays the actual measure value, with different colors representing the strategies. The graphs are ordered from most sparse (Stock) to least sparse (Adult) to compare methods for choosing the θ value at $\epsilon = 1$ ($\epsilon_1 = 0.4$, $\epsilon_2 = 0.6$). The methods include all variations described in the algorithm variation section. We note all the strategies are private except baseline 2 (orange dash line) that sets θ as the true maximum degree of the conflict graph.

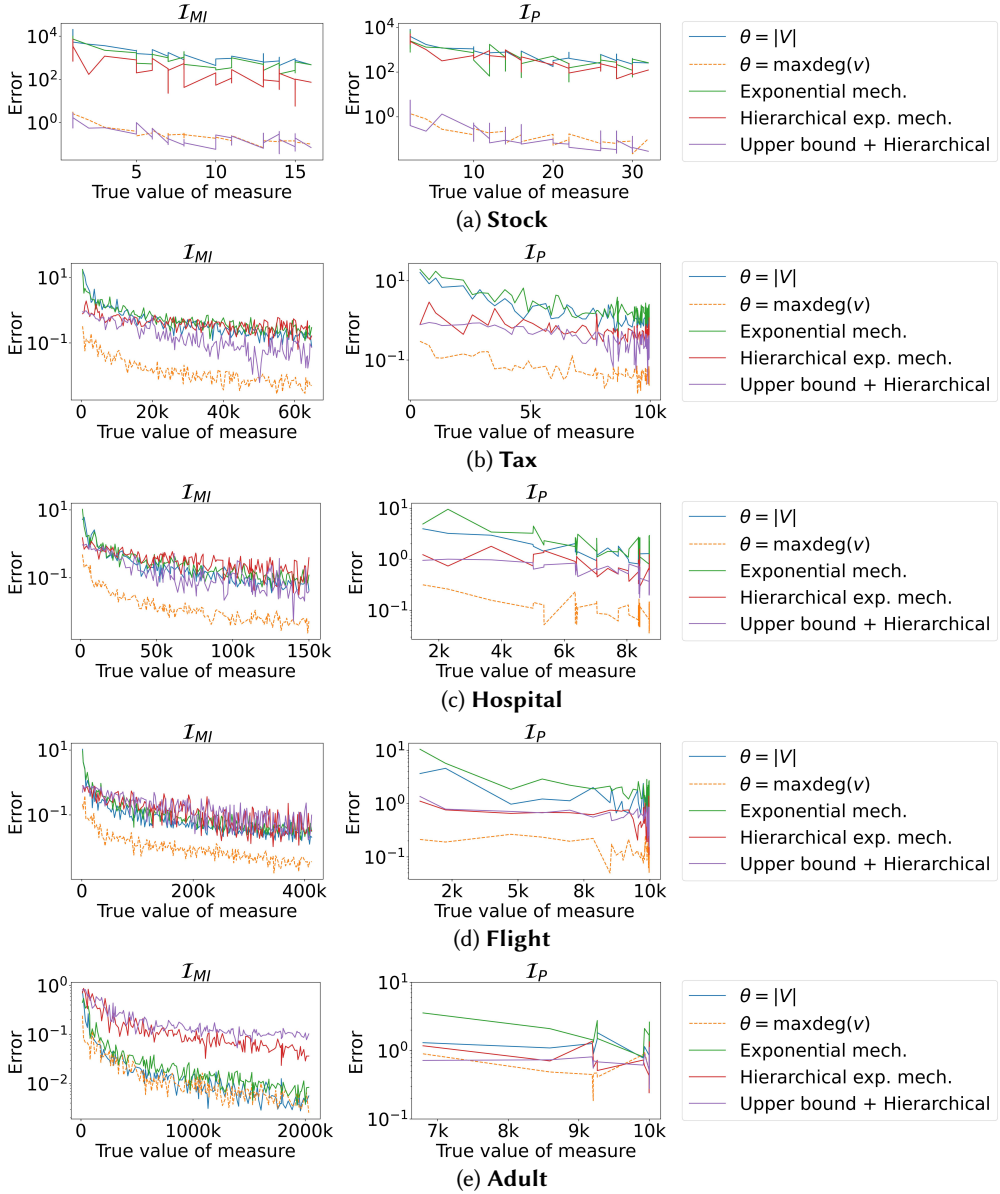


Fig. 5. Computing different strategies for choosing θ for all datasets with RNoise at $\alpha = 0.01$ and $\varepsilon = 1$. The datasets are arranged according to their densities from sparsest (top) to densest (bottom).

Our experimental results, based on error trends and graph density, reveal several key observations.

First, we consistently observed that the initial error was higher at smaller iterations across all five datasets and inconsistency measures. This is because, at smaller iterations, the true value of the measures is small due to fewer violations, and the privacy noise dominates the signal of true value.

Second, for the sparsest dataset (Stock), all strategies have errors of magnitude 3-4 larger, except the non-private baseline (orange) and our approach using both upper bound and hierarchical

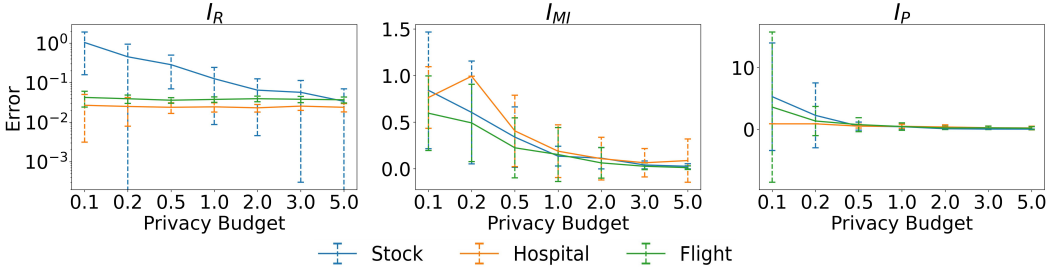


Fig. 6. Computing inconsistency measures for different datasets with RNoise at $\alpha = 0.005$ and varying privacy budgets. I_R plot has a y-axis in the log scale.

exponential mechanism (purple). This is because the candidate set contains many large candidates, and it is crucial to prune it using the upper bound strategy to get meaningful results.

Third, for the moderately sparse graphs (Tax and Hospital), our approach consistently (purple) consistently outperformed other private methods. However, the two-step hierarchical exponential mechanism (red), which had a 3 magnitude higher error for Stock, demonstrated comparable performance within a 1-magnitude error difference for Tax and Hospital. This suggests that when the true max degree is not excessively low, estimating it without the upper bound strategy can be effective.

Finally, for the densest graphs (Flight and Adult), we observe that the optimized exponential mechanisms (red and purple) outperform the private baselines (blue and green) for the I_P measure (nodes with positive degree) plots (above). However, they fail to beat even the naive baseline (blue) for the I_{MI} (number of edges) measure (below). This is because the optimal degree bound value I_{MI} over the dense graphs is close to the largest possible value $|V|$. For such a case, our optimized EM is not able to prune too many candidates and lower the sensitivity, and hence, it wastes some of the privacy budget in the pruning process. However, the relative errors of all the algorithms are reasonably small for dense graphs, and the noisy answers do preserve the order of the true measures (shown in previous experiments in Figures 3 and 4).

Varying privacy budget. Figure 6 illustrates how our algorithms perform at $\epsilon \in [0.1, 0.2, 0.5, 1.0, 2.0, 3.0, 5.0]$ with varying privacy budgets. The rightmost figure for the repair measure I_R has a log scale on the y-axis for better readability. We experiment with three datasets of different density properties (sparsest Stock and densest Flight) and show that our algorithm gracefully scales with the ϵ privacy budget for all three inconsistency measures. We also observe that the algorithm has a more significant error variation at smaller epsilons (< 1) except for Stock, which has a larger variation across all epsilons. This happens when the true value for this measure is small, and adding noise at a smaller budget ruins the estimate drastically. For the I_R measure, the private value reaches with 0.05 error at $\epsilon = 3$ for Stock and as early as $\epsilon = 0.1$ for others.

Runtime and scalability analysis. Figure 7 presents the runtime of our methods for each measure. We fix the privacy budget $\epsilon = 1$ and run three experiments by varying the graph size, numbers of DCs, and dataset. For the first experiment, we use our largest dataset, Tax, and vary the number of nodes from 10^2 to 10^6 . RNoise uses $\alpha = 0.005$ in the left plot and $\alpha = 0.01$ in the center plot. We observe that the number of edges scales exponentially when we increase the number of nodes, and the time taken by our algorithm is proportional to the graph size. With a graph of 10^2 nodes and ≤ 10 edges, our algorithm takes 10^{-3} seconds and goes up to 4500 seconds with 10^6 nodes and 322 million edges. We omit the experiment with 10^6 nodes and $\alpha = 0.01$ as the graph size for this

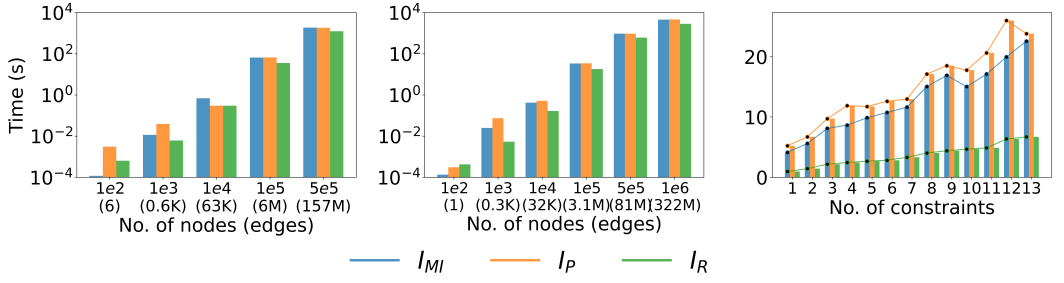


Fig. 7. Runtime analysis for all measures. Varying graph size on Tax dataset with RNoise $\alpha = 0.005$ (left) and $\alpha = 0.01$ (center) and varying #DCs for Flight dataset (right). I_{MI} and I_P plots have y-axis in the log scale.

experiment went over 30GB and was not supported by the pickle library we use to save our graph. This is not an artifact of our algorithm and can be scaled in the future using other graph libraries.

For our second experiment, we choose a subset of 10k rows of the Flight dataset and vary the number of DCs to 13 with $\alpha = 0.005$. With one DC and $\alpha = 0.005$, our algorithm takes approximately 5 seconds for I_{MI} and I_P and ≤ 1 second for I_R , and goes up to 25 seconds and 5 seconds, respectively, for $\alpha = 0.065$ and 13 DCs. We also notice some dips in the trend line (e.g., at 10 and 13 constraints) because the exponential mechanism chooses larger thresholds at those points, and the edge addition algorithm takes slightly longer with chosen thresholds. Our third experiment on varying datasets behaves similarly and is deferred to Appendix A.4 in the full version [47] for lack of space.

7 Related Work

We survey relevant works in the fields of DP and data repair.

Differential privacy. DP has been studied in multiple settings [29, 29, 35, 45, 58, 62], including systems that support complex SQL queries that, in particular, can express integrity constraints [14]. The utilization of graph databases under DP has also been thoroughly explored, with both edge privacy [27, 30–32, 56, 65] and node privacy [7, 12, 33]. Our approach draws on [12] to allow efficient DP computation of the inconsistency measures over the conflict graph. In contrast, we have seen worse performance from alternative approaches for releasing graph statistics that tend to truncate edges or nodes aggressively. In the context of data quality, previous work [37] has proposed a framework for releasing a private version of a database relation for publishing, supporting specific repair operations, while more recent, work [21] provides a DP synthetic data generation mechanism that considers soft DCs [9].

Data repair. Various classes of constraints were proposed in the literature, including FDs, conditional FDs [8], metric FDs [36], and DCs [9]. We focused on DCs, a general class of integrity constraints that subsumes the aforementioned constraints. While computing the minimal data repair in some cases has been shown to be polynomial time [43], computing the minimal repair in most general cases, corresponding to I_R , is NP-hard. Therefore, a prominent vein of research has been devoted to utilizing these constraints for data repairing [2, 5, 11, 20, 22, 23, 43, 54]. The repair model in these works varies between several options: tuple deletion, cell value updates, tuple addition, and combinations thereof. The process of data repairing through such algorithms can be time consuming due to the size of the data and the size and complexity of the constraint set. Hence, previous work [40, 44] proposed to keep track of the repairing process and ensure that it progresses correctly using inconsistency measures. In our work, we capitalize on the suitability of these measures to DP as they provide an aggregate form that summarizes the quality of the data for a given set of constraints.

8 Future Work

This paper analyzes a novel problem of computing inconsistency measures privately. There are many interesting directions for the continuation of this work. This paper shows a naive threshold bound for general DCs that can be improved for better performance of our algorithm. Our proposed conflict graphs algorithm is intractable for the $\mathcal{I}_D$ and $\mathcal{I}_{MC}$ measures that other heuristic or approximation-based approaches may solve in the future. The vertex cover size algorithm using the stable ordering of edges is general purpose and may be used in other problems outside of inconsistency measures. It can also be analyzed further in future work to return the vertex cover set. Another interesting direction is to develop these measures in a multi-relational database setting. Our approach can be extended to multi-relational tables as long as we can create conflict graphs representing the violations. However, in the multi-table setting, we must consider additional constraints that require tackling several challenges. In particular, these challenges may arise when we have non-binary or non-anti-monotonic constraints. Non-binary constraints with more than two tuples participating in a constraint lead to hypergraphs, and constraints like foreign key and inclusion constraints are non-anti-monotonic. Thus, they cannot be represented as conflict graphs, and as such, are outside the scope of our work. Furthermore, in the context of DP, constraints on multi-relational tables also have implications for defining neighboring datasets and sensitivity that must be carefully considered. Our future work also includes studying the problem of private inconsistency measures with different privacy notions, such as k-anonymity, and using these private inconsistency measures in real-world data cleaning applications.

9 Conclusions

We proposed a new problem of inconsistency measures for private datasets in the DP setting. We studied five measures and showed that two are intractable with DP, and the others face a significant challenge of high sensitivity. To solve this challenge, we leveraged the dataset's conflict graph and used graph projection and a novel approximate DP vertex cover size algorithm to accurately estimate the private inconsistency measures. We found that parameter selection was a significant challenge and were able to overcome it using optimization techniques based on the constraint set. To test our algorithm, we experimented with five real-world datasets with varying density properties and showed that our algorithm could accurately calculate these measures across all datasets.

Acknowledgments

Shubhankar Mohapatra was supported by an Ontario graduate, vector graduate research, and David R. Cheriton scholarships. The work of Xi He was supported by NSERC through a Discovery Grant, an alliance grant, and the Canada CIFAR AI Chairs program. The work of Amir Gilad was funded by the Israel Science Foundation (ISF) under grant 1702/24, the Scharf-Ullman Endowment, and the Alon Scholarship.

References

- [1] John M Abowd. 2018. The US Census Bureau adopts differential privacy. In *SIGKDD*. 2867–2867.
- [2] Foto N. Afrati and Phokion G. Kolaitis. 2009. Repair Checking in Inconsistent Databases: Algorithms and Complexity. In *ICDT*. 31–41.
- [3] Barry Becker and Ronny Kohavi. 1996. Adult. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>.
- [4] Leopoldo E. Bertossi. 2018. Measuring and Computing Database Inconsistency via Repairs. In *SUM (LNCS, Vol. 11142)*. Springer, 368–372.
- [5] Leopoldo E. Bertossi, Solmaz Kolahi, and Laks V. S. Lakshmanan. 2013. Data Cleaning and Query Answering with Matching Dependencies and Matching Functions. *Theory Comput. Syst.* 52, 3 (2013), 441–482.
- [6] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient Denial Constraint Discovery with Hydra. *Proc. VLDB Endow.* 11, 3 (2017), 311–323.
- [7] Jeremiah Blocki, Avrim Blum, Anupam Datta, and Or Sheffet. 2013. Differentially private data analysis of social networks via restricted sensitivity. In *Proceedings of the 4th conference on Innovations in Theoretical Computer Science*. 87–96.
- [8] Philip Bohannon, Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2007. Conditional functional dependencies for data cleaning. In *ICDE*.
- [9] Jan Chomicki and Jerzy Marcinkowski. 2005. Minimal-change integrity maintenance using tuple deletions. *Inf. Comput.* 197, 1-2 (2005), 90–121.
- [10] Xu Chu, Ihab F Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *Proceedings of the VLDB Endowment* 6, 13 (2013), 1498–1509.
- [11] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *ICDE*. 458–469.
- [12] Wei-Yen Day, Ninghui Li, and Min Lyu. 2016. Publishing graph degree distribution with node differential privacy. In *SIGMOD*. 123–138.
- [13] Bolin Ding, Janardhan Kulkarni, and Sergey Yekhanin. 2017. Collecting telemetry data privately. *Advances in Neural Information Processing Systems* 30 (2017).
- [14] Wei Dong, Juanru Fang, Ke Yi, Yuchao Tao, and Ashwin Machanavajjhala. 2022. R2t: Instance-optimal truncation for differentially private query evaluation with foreign keys. In *SIGMOD*. 759–772.
- [15] Cynthia Dwork. 2006. Differential Privacy. In *ICALP*, Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.), Vol. 4052. 1–12.
- [16] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. 2006. Our Data, Ourselves: Privacy Via Distributed Noise Generation. In *EUROCRYPT*, Serge Vaudenay (Ed.), Vol. 4004. Springer, 486–503.
- [17] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*. Springer, 265–284.
- [18] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam D. Smith. 2016. Calibrating Noise to Sensitivity in Private Data Analysis. *J. Priv. Confidentiality* 7, 3 (2016), 17–51. <https://doi.org/10.29012/jpc.v7i3.405>
- [19] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. 2014. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 1054–1067.
- [20] Ronald Fagin, Benny Kimelfeld, and Phokion G. Kolaitis. 2015. Dichotomies in the Complexity of Preferred Repairs. In *PODS*. 3–15.
- [21] Chang Ge, Shubhankar Mohapatra, Xi He, and Ihab F. Ilyas. 2021. Kamino: Constraint-Aware Differentially Private Data Synthesis. *Proc. VLDB Endow.* 14, 10 (2021), 1886–1899. <https://doi.org/10.14778/3467861.3467876>
- [22] Floris Geerts, Giansalvatore Mecca, Paolo Papotti, and Donatello Santoro. 2013. The LLUNATIC Data-Cleaning Framework. *Proc. VLDB Endow.* 6, 9 (2013), 625–636.
- [23] Amir Gilad, Daniel Deutch, and Sudeepa Roy. 2020. On Multiple Semantics for Declarative Database Repairs. In *SIGMOD*. 817–831.
- [24] John Grant and Anthony Hunter. 2013. Distance-Based Measures of Inconsistency. In *ECSQARU (LNCS, Vol. 7958)*. Springer, 230–241.
- [25] John Grant and Anthony Hunter. 2023. Semantic inconsistency measures using 3-valued logics. *Int. J. Approx. Reason.* 156 (2023), 38–60.
- [26] Anupam Gupta, Katrina Ligett, Frank McSherry, Aaron Roth, and Kunal Talwar. 2010. Differentially private combinatorial optimization. In *Proceedings of the twenty-first annual ACM-SIAM symposium on Discrete Algorithms*. SIAM, 1106–1125.
- [27] Michael Hay, Chao Li, Gerome Miklau, and David Jensen. 2009. Accurate estimation of the degree distribution of private networks. In *2009 Ninth IEEE International Conference on Data Mining*. IEEE, 169–178.

- [28] Justin Hsu, Aaron Roth, Tim Roughgarden, and Jonathan Ullman. 2014. Privately solving linear programs. In *ICALP*. Springer, 612–624.
- [29] Noah Johnson, Joseph P Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539.
- [30] Vishesh Karwa, Sofya Raskhodnikova, Adam Smith, and Grigory Yaroslavtsev. 2011. Private analysis of graph structure. *Proceedings of the VLDB Endowment* 4, 11 (2011), 1146–1157.
- [31] Vishesh Karwa and Aleksandra B Slavković. 2012. Differentially private graphical degree sequences and synthetic graphs. In *Privacy in Statistical Databases: UNESCO Chair in Data Privacy, International Conference, PSD 2012, Palermo, Italy, September 26–28, 2012. Proceedings*. Springer, 273–285.
- [32] Vishesh Karwa, Aleksandra B Slavković, and Pavel Krivitsky. 2014. Differentially private exponential random graphs. In *Privacy in Statistical Databases: UNESCO Chair in Data Privacy, International Conference, PSD 2014, Ibiza, Spain, September 17–19, 2014. Proceedings*. Springer, 143–155.
- [33] Shiva Prasad Kasiviswanathan, Kobbi Nissim, Sofya Raskhodnikova, and Adam D. Smith. 2013. Analyzing Graphs with Node Differential Privacy. In *TCC*, Amit Sahai (Ed.), Vol. 7785. Springer, 457–476. https://doi.org/10.1007/978-3-642-36594-2_26
- [34] Benny Kimelfeld, Ester Livshits, and Liat Peterfreund. 2020. Counting and enumerating preferred database repairs. *Theor. Comput. Sci.* 837 (2020), 115–157. <https://doi.org/10.1016/j.tcs.2020.05.016>
- [35] Ios Kotsogiannis, Yuchao Tao, Xi He, Maryam Fanaeepour, Ashwin Machanavajjhala, Michael Hay, and Jerome Miklau. 2019. Privatesql: a differentially private sql query engine. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1371–1384.
- [36] Nick Koudas, Avishek Saha, Divesh Srivastava, and Suresh Venkatasubramanian. 2009. Metric functional dependencies. In *ICDE*.
- [37] Sanjay Krishnan, Jiannan Wang, Michael J. Franklin, Ken Goldberg, and Tim Kraska. 2016. PrivateClean: Data Cleaning and Differential Privacy. In *SIGMOD*. ACM, 937–951. <https://doi.org/10.1145/2882903.2915248>
- [38] M. Lichman. 2013. UCI machine learning repository. <https://archive.ics.uci.edu/ml/datasets/adult>.
- [39] Jinfei Liu, Jian Lou, Junxu Liu, Li Xiong, Jian Pei, and Jimeng Sun. 2021. Dealer: an end-to-end model marketplace with differential privacy. *Proceedings of the VLDB Endowment* 14, 6 (2021).
- [40] Ester Livshits, Leopoldo E. Bertossi, Benny Kimelfeld, and Moshe Sebag. 2020. The Shapley Value of Tuples in Query Answering. In *ICDT*, Vol. 155. 20:1–20:19.
- [41] Ester Livshits, Alireza Heidari, Ihab F. Ilyas, and Benny Kimelfeld. 2020. Approximate Denial Constraints. *Proc. VLDB Endow.* 13, 10 (2020), 1682–1695.
- [42] Ester Livshits and Benny Kimelfeld. 2022. The Shapley Value of Inconsistency Measures for Functional Dependencies. *Log. Methods Comput. Sci.* 18, 2 (2022).
- [43] Ester Livshits, Benny Kimelfeld, and Sudeepa Roy. 2020. Computing Optimal Repairs for Functional Dependencies. *ACM Trans. Database Syst.* 45, 1 (2020), 4:1–4:46. <https://doi.org/10.1145/3360904>
- [44] Ester Livshits, Rina Kochirgan, Segev Tsur, Ihab F. Ilyas, Benny Kimelfeld, and Sudeepa Roy. 2021. Properties of Inconsistency Measures for Databases. In *SIGMOD*. 1182–1194.
- [45] Ryan McKenna, Jerome Miklau, Michael Hay, and Ashwin Machanavajjhala. 2018. Optimizing error of high-dimensional statistical queries under differential privacy. *Proc. VLDB Endow.* 11, 10 (2018), 1206–1219.
- [46] Frank McSherry and Kunal Talwar. 2007. Mechanism design via differential privacy. In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS'07)*. IEEE, 94–103.
- [47] Shubhankar Mohapatra, Amir Gilad, Xi He, and Benny Kimelfeld. 2025. Computing Inconsistency Measures Under Differential Privacy. *CoRR* abs/2502.11009 (2025). <https://doi.org/10.48550/ARXIV.2502.11009> arXiv:2502.11009
- [48] Department of Transportation. 2020. Flight. Research and Innovative Technology Administration. <https://data.transportation.gov/d/7fzd-cqts>.
- [49] Oleh Onyshchak. 2020. Stock Market Dataset. <https://doi.org/10.34740/KAGGLE/DSV/1054465>
- [50] Francesco Parisi and John Grant. 2019. Inconsistency measures for relational databases. *arXiv preprint arXiv:1904.03403* (2019).
- [51] Shweta Patwa, Danyu Sun, Amir Gilad, Ashwin Machanavajjhala, and Sudeepa Roy. 2023. DP-PQD: Privately Detecting Per-Query Gaps In Synthetic Data Generated By Black-Box Mechanisms. *Proc. VLDB Endow.* 17, 1 (2023), 65–78.
- [52] Eduardo H. M. Pena, Eduardo Cunha de Almeida, and Felix Naumann. 2021. Fast Detection of Denial Constraint Violations. *Proc. VLDB Endow.* 15, 4 (2021), 859–871.
- [53] CORGIS Dataset Project. [n. d.]. Hospitals CSV File. <https://corgis-edu.github.io/corgis/csv/hospitals/>.
- [54] Theodoros Rekatinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *PVLDB* 10, 11 (2017), 1190–1201.
- [55] Dan Roth. 1993. On the Hardness of Approximate Reasoning. In *IJCAI*. Morgan Kaufmann, 613–619.
- [56] Alessandra Sala, Xiaohan Zhao, Christo Wilson, Haitao Zheng, and Ben Y Zhao. 2011. Sharing graphs using differentially private graph models. In *Proceedings of the 2011 ACM SIGCOMM conference on Internet measurement conference*. 81–98.

- [57] Peng Sun, Xu Chen, Guocheng Liao, and Jianwei Huang. 2022. A Profit-Maximizing Model Marketplace with Differentially Private Federated Learning. In *INFOCOM*. IEEE, 1439–1448.
- [58] Yuchao Tao, Xi He, Ashwin Machanavajjhala, and Sudeepa Roy. 2020. Computing local sensitivities of counting queries with joins. In *SIGMOD*. 479–494.
- [59] Matthias Thimm. 2017. On the compliance of rationality postulates for inconsistency measures: A more or less complete picture. *KI-Künstliche Intelligenz* 31 (2017), 31–39.
- [60] Zhihua Tian, Jian Liu, Jingyu Li, Xinle Cao, Ruoxi Jia, and Kui Ren. 2022. Private Data Valuation and Fair Payment in Data Marketplaces. *CoRR* abs/2210.08723 (2022).
- [61] Vijay V Vazirani. 1997. Approximation algorithms. *Georgia Inst. Tech* (1997).
- [62] Royce J Wilson, Celia Yuxin Zhang, William Lam, Damien Desfontaines, Daniel Simmons-Marengo, and Bryant Gipson. 2019. Differentially private sql with bounded user contribution. *arXiv preprint arXiv:1909.01917* (2019).
- [63] Mingyan Xiao, Ming Li, and Jie Jennifer Zhang. 2023. Locally Differentially Private Personal Data Markets Using Contextual Dynamic Pricing Mechanism. *IEEE Trans. Dependable Secur. Comput.* 20, 6 (2023), 5043–5055.
- [64] Liyang Xie, Kaixiang Lin, Shu Wang, Fei Wang, and Jiayu Zhou. 2018. Differentially Private Generative Adversarial Network. *CoRR* abs/1802.06739 (2018). arXiv:1802.06739 <http://arxiv.org/abs/1802.06739>
- [65] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2015. Private release of graph statistics using ladder functions. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 731–745.
- [66] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2017. Privbayes: Private data release via bayesian networks. *ACM Transactions on Database Systems (TODS)* 42, 4 (2017), 1–41.
- [67] Jun Zhang, Xiaokui Xiao, and Xing Xie. 2016. PrivTree: A Differentially Private Algorithm for Hierarchical Decompositions. In *SIGMOD*. 155–170.

Received October 2024; revised January 2025; accepted February 2025